A Review of Reinforcement Learning Applications for Monetization-Aware Content Ranking in Large-Scale Recommendation Systems

ARMANDO ORDORICA, University of Toronto, Canada
YURI LAWRYSHYN, University of Toronto, Canada

Balancing advertisements with organic content in large-scale recommendation engines is a well-known challenge in the industry. While ads are essential for revenue generation in platforms that integrate them alongside organic content, they often underperform in user engagement compared to organic content. The key challenges in achieving this balance include ad selection, ranking, and integration with organic content. These involve decisions about whether to insert an ad, which ad to select, and where to place it.

Traditional methods optimize the probability of immediate ad clickthrough with fixed placements and are typically framed as supervised learning problems. However, these approaches estimate conditional expectations of immediate outcomes under a historical logging policy, treating decisions as independent and optimizing short-horizon objectives. A more principled formulation that accounts for contextual information and policy-induced state evolution, and that optimizes long-horizon value over action sequences to jointly balance engagement and revenue, remains an open research challenge.

Reinforcement Learning (RL) provides a principled framework for modeling recommendation as a sequential decision process under uncertainty. However, implementing RL in large-scale recommendation systems faces challenges including high-dimensional state and action spaces, the risk of poorly chosen reward functions and unsafe exploration, high computational costs, and the difficulty of interpreting online policy learning.

This survey synthesizes RL methodologies and frameworks for jointly optimizing ads and organic content, organizing prior approaches according to how they model state, action, reward, and policy. Our review indicates that while fully online RL is theoretically appealing, documented industrial systems more frequently adopt hybrid architectures that leverage supervised learning for robust short-term estimation while incorporating RL components for long-horizon value optimization. Furthermore, many large-scale deployments rely on constrained RL formulations, such as contextual bandits and offline replay, that balance sequential optimization with practical considerations of scalability, safety, and interpretability.

Additional Key Words and Phrases: Reinforcement Learning, Ad Policy Optimization, Contextual Bandits, Recommendation Systems, Off-Policy Learning, Online Advertising, Deep Reinforcement Learning, Multi-Objective Optimization

Compiled: 2026-03-15 17:37 EDT

1 INTRODUCTION

Platforms such as Instagram, TikTok, and Pinterest develop and deploy large-scale content recommendation systems whose revenue models depend on jointly optimizing monetization and long-term engagement in interactive environments [Danescu-Niculescu-Mizil et al. 2010; Zhao et al. 2020].

Authors' addresses: Armando Ordorica, University of Toronto, Toronto, Canada; Yuri Lawryshyn, University of Toronto, Toronto, Canada.

2025. 0360-0300/2025/3-ART $15.00
https://doi.org/10.1145/nnnnnnn.nnnnnnn

Failing to achieve an adequate balance can result in issues such as ad fatigue, where users become less responsive to ads due to overexposure, and user churn, where users discontinue using the platform [Sagtani et al. 2023; Silberstein et al. 2023].

Jointly optimizing revenue and long-term engagement is structurally difficult for several fundamental reasons. First, the monetization and engagement objectives can conflict: actions that increase immediate monetization, such as higher ad load, may degrade future engagement and retention, ultimately reducing long-term revenue [Sagtani et al. 2023; Yan et al. 2020]. Second, immediate per-interaction signals are directly observable and easier to optimize, whereas long-term outcomes unfold over extended horizons and are harder to measure and attribute to specific actions [Hohnhold et al. 2015b; McDonald et al. 2023b; Wang et al. 2022]. Third, recommendation decisions alter user state dynamics, creating feedback loops between engagement and future monetization capacity [Ie et al. 2019c; Zou et al. 2019]. These properties make the trade-off inherently sequential and policy-dependent.

To address these challenges, large-scale platforms have relied on supervised deep learning models optimized for short-term engagement signals such as clicks, dwell time, and conversions [Cheng et al. 2016; Covington et al. 2016; McMahan et al. 2013; Yi et al. 2014]. By learning rich representations of users, items, and context from large-scale interaction logs, these models estimate conditional expectations of immediate outcomes and use these predictions to guide content selection and ranking [Cheng et al. 2016; Covington et al. 2016]. This paradigm has delivered substantial improvements in short-term engagement and click-through prediction performance [Cheng et al. 2016; Covington et al. 2016; McMahan et al. 2013]. It remains central to modern industrial recommender systems.

While neural networks remain powerful function approximators, optimizing supervised objectives on historical logged data does not directly address the sequential and interactive nature of the system. In such settings, SL is structurally insufficient for two key reasons. First, SL ignores policy-induced distribution shifts. It optimizes conditional expectations of immediate outcomes (e.g., $\mathbb{E}[r_t \mid s_t, a_t]$) using data generated under a historical policy. This implicitly assumes that the training distribution is fixed. In interactive systems, however, the recommendation policy shapes user behavior, and user behavior in turn determines future training data. This violates the stationarity assumptions underlying standard supervised optimization [Dudík et al. 2011; Swaminathan and Joachims 2015]. Second, SL lacks a principled mechanism for evaluating counterfactual trajectories as it relies exclusively on realized trajectories. Because user journeys are highly personalized and path-dependent, most possible interaction sequences are never observed in logged data. Long-term outcomes are therefore not fixed labels, but functions of the policy that generates future trajectories [Agarwal and Chen 2016].

RL provides a framework in recommendation systems to solve the two problems above [Afsar et al. 2022]. First, RL addresses policy-induced distribution shifts by explicitly modeling recommendation as a Markov Decision Process (MDP) in which actions influence future states [Puterman 2014; Sutton and Barto 2018] . Second, RL enables policy evaluation and improvement over complete trajectories through optimization of the expected cumulative return under policy-induced dynamics [Sutton and Barto 2018]. Rather than replacing SL, RL can be viewed as embedding supervised prediction within a sequential control framework [Afsar et al. 2022; Sutton and Barto 2018]. Deep models remain central for representation learning and reward estimation, while RL augments them with the machinery required to optimize policy-dependent, long-term objectives [Chen et al. 2022; Dulac-Arnold et al. 2019; Levine et al. 2020].

Despite the promising theoretical framework that RL offers for optimizing long-term value in recommendation systems, implementing RL in real-world, large-scale platforms remains complex [Dulac-Arnold et al. 2021]. First, the state space, which reflects the current context of the user

and the environment, and the action space, which captures how the recommender system can be tuned, are both essentially infinite [Zhu et al. 2021]. This vastness makes learning and optimization computationally intensive and requires scalable approximation methods, as exact dynamic programming over such state and action spaces is infeasible in practice [Afsar et al. 2022]. Second, designing appropriate reward functions is critical, as poorly chosen reward functions can lead to unintended system behaviors that degrade user experience or negatively affect business revenue [Everitt et al. 2021]. For example, a reward function that overly prioritizes click-based metrics may drive the system to promote clickbait content, boosting short-term engagement but eroding user trust and satisfaction over time [Wu et al. 2017]. Conversely, a reward function too heavily skewed toward revenue metrics could result in excessive ad load, degrading the user experience and accelerating churn. Striking the right balance in reward function design is essential to achieve both user engagement and monetization objectives without compromising the platform's long-term goals. Furthermore, interpretability is a major challenge in RL deployment. Understanding the learned policies can be particularly difficult when policies are updated in real-time during user interactions (online policy learning) [Dulac-Arnold et al. 2019; Glanois et al. 2024; Vouros 2022]. In extreme cases, this lack of transparency could result in non-compliant or unethical behavior, posing significant risks to both users and the business [Lambert et al. 2023].

In light of the challenges outlined above, this paper provides a synthesis of RL approaches to monetization-aware recommendation systems through the lens of the core components of a Markov Decision Process. Although previous surveys address RL in recommendation systems [Afsar et al. 2022; Lin et al. 2023], our systematic literature search (see Section 2) did not identify any survey that explicitly focuses on the **joint optimization of ads and organic content in sequential and interactive environments**, which leaves limited guidance for systems that must jointly reason about monetization and long-term user value. This paper addresses this limitation by reviewing approaches that move beyond supervised objectives defined under fixed logging policies to analyze how RL agents model and optimize sequential trade-offs between revenue and engagement.

This survey introduces component-level taxonomy that clarifies how existing systems operationalize state, action, reward, and policy, and uses it to systematically organize and analyze more than 150 academic and industrial works.

The paper begins with a historical overview of recommendation systems with monetization components, tracing the evolution from rule-based and heuristic approaches to tree-based models and modern deep learning architectures. This progression highlights the limitations of short-horizon optimization and motivates reinforcement learning as a framework for modeling long-horizon, policy-dependent objectives. The remainder of the paper is structured around these components (state, action, reward, and policy) using them as analytical categories to compare and contrast existing systems. Rather than structuring prior work chronologically or by algorithmic family, we compare systems according to how they define and operationalize these components. A minimal RL environment (FrozenLake) is used as a conceptual scaffold to introduce the abstraction before mapping it to large-scale monetization-aware recommender systems.

2 PAPER COLLECTION METHODOLOGY

To ensure broad coverage, relevant papers were collected from both academic and industry sources. Foundational academic works such as *Reinforcement Learning: An Introduction* [Sutton and Barto 2018], *Statistical Methods for Recommender Systems* [Agarwal and Chen 2016], and *Artificial Intelligence: A Modern Approach* [Russell and Norvig 2016] were used to establish key mathematical foundations, including Markov Decision Processes (MDPs), Contextual Bandits, Off-Policy Learning, and Offline Replay.

To reflect the internal perspective of a production-facing ad policy team, a curated list of 10–15 papers referenced within Pinterest's Ad Policy team was also compiled. Examples include *Jointly Learning to Recommend and Advertise* [Zhao et al. 2020], *DEAR: Deep Reinforcement Learning for Online Advertising Impressions in Recommender Systems* [Zhao et al. 2021], *Ad-load Balancing via Off-Policy Learning* [Sagtani et al. 2024], and *Practical Bandits* [van den Akker et al. 2024]. In addition, targeted keyword searches such as "Reinforcement Learning ads [company name]" and "Contextual Bandit [company name]" were conducted for leading companies in the advertising and recommendation space (e.g., Google, Meta, Pinterest, Microsoft, TikTok, YouTube, LinkedIn), surfacing additional papers describing real-world systems and methodologies. To complement those efforts, a separate search was conducted on Google Scholar for relevant review papers. While no surveys were found that specifically focused on RL in ad recommendation systems, related reviews on "contextual bandits", "ad fatigue", and "recommender systems" were used to identify additional references.

Finally, manual crawling of citation networks in cornerstone papers was used to identify additional key contributions. For instance, tracing citations from *Jointly Learning to Recommend and Advertise* [Zhao et al. 2020], *DEAR* [Zhao et al. 2021], and *Offline Reinforcement Learning* [Levine et al. 2020] led to works such as *Ads Allocation in Feed via Constrained Optimization* [Yan et al. 2020] and *A Contextual Bandit Bake-off* [Bietti et al. 2021]. In total, 153 papers were selected and reviewed. Papers were ranked according to three main criteria: influence, to capture the foundational or widely adopted ideas in the field; industry relevance, to ensure that the outcomes were sound, implementable, and actionable; and recency, to reflect the latest methodological developments and deployment practices.

While recent and highly cited work was prioritized, papers introducing novel methodologies were also included regardless of citation count. To contextualize the selected works, this survey builds from the ground up. Each component of the Markov Decision Process, including states, actions, rewards, and policies, is defined and illustrated using a toy example such as the Frozen Lake. This foundation is then extended to practice through a review of how each component is operationalized in industry, along with a discussion of the limitations of those approaches. Drawing on existing methods and lessons from industry, the resulting framework is intended to inform practitioners exploring how RL can be applied in their organizations to develop ad policies that balance monetization with organic engagement. A comparison between the toy example and real-world implementation parallels can be found in Table 1.

3 A BRIEF HISTORY OF ADVERTISING AND RECOMMENDATION SYSTEMS: FROM RULE-BASED TO DEEP LEARNING-BASED JOINT OPTIMIZATION

This section traces the evolution of advertising recommendation algorithms, culminating in the adoption of RL to overcome the limitations of earlier approaches. It begins by outlining how early online advertising primarily mirrored offline methods through banner ads and other static formats. It continues by explaining how static online advertising evolved over time to incorporate rule-based models and more sophisticated heuristic techniques. The discussion then transitions to the integration of machine learning methods, including regression-based models, tree-based algorithms, and collaborative filtering, followed by the emergence of deep learning and embedding-based approaches that effectively capture the nonlinear interactions between users and content. Finally, RL is introduced as a promising solution to address the shortcomings of preceding methodologies. This evolution is visually represented in Figure 1, which shows the initial adoption and popularization of techniques and methodologies in online advertising.

MDP Component	FrozenLake Example	Ad Recommendations
Environment	The frozen lake.	The users.
Goal	Reach the goal (G) from the start position (S) while avoiding holes (H). The agent must avoid falling into a hole, and reaching the goal faster is rewarded.	Maximize revenue without compromising user experience.
Episode	A game.	A time period (T), typically 2 weeks, corresponding to an A/B experiment where enough data is collected to evaluate policy performance across users.
Rewards	Rewarded for reaching the goal (i.e., +100), but penalized for delays (i.e., -1 per step).	The reward is typically represented by a utility function, which could be a weighted sum of metrics like user engagement (clicks, time spent) and revenue generated (ad clicks, conversions). The challenge lies in balancing short-term revenue with long-term user retention.
Agent	The learner.	The Recommendation System.
Actions	Up, down, left and right.	Basically infinite. But when it comes to ad recommendations specifically, some typical actions are the following: • Reserve Price Floor: The minimum price at which an ad impression is sold. • Displacement Cost Threshold: A measure of the opportunity cost for showing an ad. • Max Ad Load: The maximum number of ads that can be shown without reducing user satisfaction. • Ad Quality Thresholds: Standards used to ensure ads meet a quality bar before being shown. • Ad Placement: The specific position of ads within the user feed (e.g., first, third, or fifth position).
States	All possible grid positions the agent can occupy, representing the state space of the environment.	Signals: • User Context: Demographics, user profile • Interaction Data: Previous interactions with ads, organic content • Contextual Features: Time of day, browsing context • Embeddings: Latent feature representations of user interests or ad features
Policy	A mapping of grid positions to optimal action to take for a maximum reward.	A mapping of user segments to ad treatments to maximize both revenue and user experience. This policy can be dynamically adjusted based on real-time feedback or offline simulations.

Table 1. Comparison of Frozen Lake Example and Large-Scale Recommendation Systems for Ad Policy Optimization

Table 2 summarizes three distinct but interrelated evolutionary tracks in ad optimization: (1) pricing and objectives, (2) representation, and (3) sequential horizon of decision-making. These dimensions evolved somewhat in parallel rather than strictly sequentially, and modern systems typically combine advances from all three tracks. This framework helps practitioners identify which

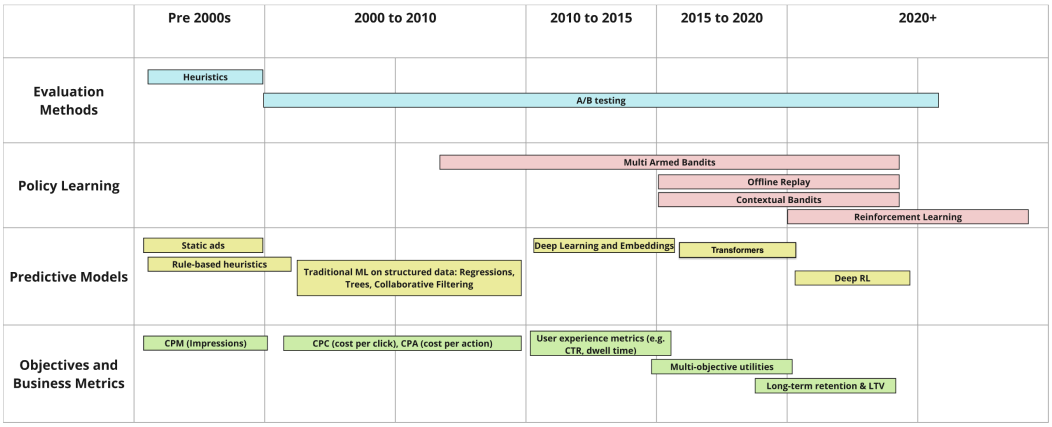

Fig. 1. Timeline Showing Initial Adoption and Popularization of Techniques and Methodologies in Online Advertising

Note: The boundaries and lengths of rectangles are approximate. Innovations and transitions between techniques and objectives occurred gradually and continuously, rather than sharply at specific dates.

dimension requires improvement for their specific business context. Additionally, Table 3 describes complementary evaluation methods used to validate and infer policy performance.

Method	Objective	Use Case
A/B Testing	Validate experiments; causal attribution	New model launches; feature comparisons; statistical significance required
Offline Replay	Infer policy performance from logs; counterfactual estimation	Policy evaluation before deployment; large parameter search; risk-averse exploration

Table 3. Complementary Evaluation Methods. A/B testing validates experiments online, while Offline Replay enables inference on logged data. Both can be used alongside any decision-making approach in Track 3.

3.1 From Rule-Based Advertising to Traditional Machine Learning

Around the early 2000s, online advertising largely mirrored offline practices, heavily relying on non-targeted formats such as banner ads [Kant 2021]. These generic advertising methods conflicted with the objective of search engines of efficiently directing users to relevant content [Fain and Pedersen 2006]. As a marginal improvement on nontargeted ad formats like banner ads, advertisers began to utilize simple `if-then` logic with predefined rules to select and display ads based on basic criteria like user demographics. While rule-based systems were straightforward to implement, their lack of flexibility and personalization became evident as ad fatigue, defined as reduced user engagement due to repeated exposure to the same advertisement, persisted [Abrams and Vee 2007]. This issue underscored the necessity of not only leveraging user signals but also optimizing ad load tuning parameters, such as frequency and placement, to maintain user interest and maximize click-through rates [Abrams and Vee 2007]. For instance, researchers at LinkedIn introduced concepts like top slot - highest eligible position for an ad within the feed - and minimum gap - the required distance between consecutive ads to prevent oversaturation - as ad load tuning parameters [Yan et al. 2020]. Their research indicated that users are more likely to click on ads that are

Track 1: Evolution of Pricing & Objectives

Stage	What It Optimizes	Shortcomings	When Sufficient
CPM (Cost per Mille)	Impressions; brand exposure	No incentive for relevance; misaligned with user value	Brand awareness; guaranteed reach campaigns
CPC (Cost per Click)	Clicks; measurable engagement	Incentivizes clickbait; ignores post-click behavior	Direct response; traffic-focused goals
CPA (Cost per Action)	Conversions; purchases, sign-ups	Sparse signal; attribution challenges	E-commerce; clear conversion funnels
Utility Functions	Multi-objective (revenue + engagement + fatigue)	Weight tuning is subjective; hard to validate offline	When explicit trade-offs between objectives are required

Track 2: Evolution of Representation

Stage	How Users/Items Are Encoded	Shortcomings	When Sufficient
Demographics / Rules	Hand-crafted segments (age, gender, location)	Coarse; no personalization; ad fatigue	Small catalogs; regulatory constraints on data use
Tabular Features	Engineered features (click history, session stats)	Manual feature engineering; misses nonlinear patterns	Interpretable models required; limited compute
Embeddings (Two-tower)	Dense vectors capturing user-item similarity	Pointwise; doesn't model sequences or context shifts	Large-scale retrieval; content with rich metadata
Attention / Transformers	Contextual, sequential representations; multi-head	Complex tuning; heads vary across users/time	Sequential sessions; long-range dependencies matter

Track 3: Evolution of Sequential Decision-Making

Stage	Sequence Consideration	Shortcomings	When Sufficient
Rule-based	None; static if-then logic	No learning; manual updates; ignores user behavior	Low traffic; strict compliance; simple business rules
Supervised ML (LR, GBM, DL)	Pointwise; each decision independent	Ignores sequential dependencies; no exploration	When decisions are approximately independent
Multi-Armed Bandits	One-step; no context	No personalization; same arm for all users	Few discrete treatments; homogeneous user base
Contextual Bandits	One-step with context; special case of RL	Actions don't influence future states; fixed action set	Real-time personalization; immediate rewards suffice
Full RL	Multi-step; actions shape future states	Sample inefficiency; reward design; safety/interpretability	Extended sessions; ad fatigue; long-horizon objectives

Table 2. Three Evolutionary Tracks in Ad Optimization. Modern systems combine advances from all three tracks. Practitioners should identify which dimension is the bottleneck: Are objectives misaligned (Track 1)? Are representations too coarse (Track 2)? Or is the decision horizon too myopic (Track 3)?

spaced farther apart [Yan et al. 2020].

The evolution of ad pricing, from paying for impressions to paying for user actions, created pressure within the online advertising industry to improve relevance and performance-based metrics [Hu 2004]. Sponsored search, introduced by GoTo (later Overture), allowed advertisers to bid on

specific keywords, directly linking ads to user queries and addressing the limitations of generic advertising [Fain and Pedersen 2006]. Foundational pricing models, such as Cost per Mille (CPM), charged advertisers per 1,000 impressions and closely mirrored offline practices [Wikipedia contributors 2025]. Yahoo!, through the acquisition of Overture [Staff 2003], later introduced Cost per Click (CPC) [Ellam and Ottaviani 2003], a performance-based pricing model that shifted costs to reflect measurable user engagement. This evolution continued with the introduction of Cost per Action (CPA), which tied fees to specific user actions beyond clicks, such as purchases or sign-ups. These changes incentivized the advertising industry to innovate, leading to the development of more relevance-focused strategies. These foundational pricing models remain integral to the utility functions that underpin contemporary ranking algorithms [Fain and Pedersen 2006]. As advertisers increasingly demanded performance-based pricing, the ad tech industry was compelled to innovate within a competitive yet lucrative market, addressing the limitations of rule-based and heuristic models through the adoption of machine learning techniques to enable more dynamic and data-driven approaches [Zhang et al. 2018].

The next set of innovations following heuristics and performance-based targeting was led by machine learning (ML) and A/B testing [Barajas et al. 2022]. From 2010 to 2015, ad selection techniques evolved significantly with the adoption of machine learning models such as Logistic Regression, Decision Trees, and Gradient Boosting Machines [Cui et al. 2015]. By leveraging historical data and diverse user behavior signals, these models enabled more accurate predictions of user engagement [Cui et al. 2015]. In parallel, A/B testing allowed practitioners to systematically compare models and features, ensuring that any new implementation delivered measurable performance gains [Barajas et al. 2022]. Despite these successes, traditional tabular models struggled to capture nuanced user intent, particularly when processing more complex data such as images or text. In the early 2010s, deep learning gained momentum with breakthroughs like AlexNet [Krizhevsky et al. 2012], which demonstrated the power of neural networks to handle unstructured high-dimensional data. This ability to capture complex, nonlinear interactions dramatically improved ad recommendations, especially through the use of embeddings, which are dense vector representations that map words or images into high-dimensional spaces. Metrics like cosine similarity further refined personalization by aligning recommendations with user interests [Covington et al. 2016]. Embedding-based models, particularly two-tower architectures, presented a major leap forward in ad recommendation systems by effectively capturing intricate relationships between users and ads. These models utilize two neural networks: one dedicated to encoding user information and the other to encoding ad (item) information. The outputs from these networks are then combined to predict user engagement with ads [Vorotilov and Shugaepov 2023]. This approach not only boosted accuracy but also scaled effectively. Despite the significant improvements that embedding-based models introduced in terms of predictive accuracy and scalability of ad recommendation systems, the practical challenges of balancing multiple objectives, such as maximizing revenue, mitigating ad fatigue, and sustaining long-term user engagement remained unresolved.

Multi-Armed Bandits (MABs) emerged as a powerful method for online advertising around the 2010s to tackle the explore–exploit tradeoff, which consists of finding the optimal balance between choosing new ad policies with unknown performance and showing older ad policies with known positive performance [Burtini et al. 2015; Schwartz et al. 2017]. MABs are a class of algorithms for dynamic decision-making under uncertainty, originally developed in mid-20th-century probability theory [Robbins 1952]. While MABs had been widely applied in areas such as clinical trials [Thompson 1933; Villar et al. 2015], they were adopted in large-scale web and advertising platforms during the early 2010s [Chapelle and Li 2011; Li et al. 2010; Schwartz et al. 2017].

MABs allow platforms to reduce unnecessary exposure to a suboptimal treatment by continuously updating the estimates of expected reward and uncertainty of which treatment is most likely

to yield the highest reward [Kuleshov and Precup 2014]. In the context of online advertising, "treatments" are typically specific ad configurations, defined by parameters such as displacement cost, reserve price, ad load, and ad placement. "Rewards" typically represent user engagement metrics such as clicks or conversions [Schwartz et al. 2017].

In contrast to traditional A/B tests, where there are randomized treatments assigned for a predetermined amount of time, MABs dynamically learn which configuration yields the highest reward for each user at a particular time. Although this adaptive learning framework manages the trade-off between minimizing regret from suboptimal treatments and exploring potentially superior configurations, defining the precise reward function to optimize remains an open challenge [Wen et al. 2019].

To formalize multi-objective trade-offs such as balancing revenue and user satisfaction, utility functions have been employed in online advertising since the early 2000s [Engel and Chickering 2008]. Drawing on microeconomic utility theory, platforms formalized multi-objective trade-offs through scalar utility functions that encode multiple objectives — such as long term revenue and engagement — into a single metric, enabling principled comparison and optimization [Engel and Chickering 2008]. When integrated into MAB frameworks, this unified reward signal guides exploration toward policies that balance short-term performance (e.g., clicks or conversions) with long-term considerations such as user trust and fatigue.

Although utility-weighted bandits can encode long-term targets via proxy rewards, they remain structurally one-step: they estimate conditional expectations under a logging policy and do not represent policy-induced feedback loops, thereby limiting counterfactual evaluation of multi-step trajectories under alternative policies.

3.2 Advances in Deep Learning: Attention Models, Transformers, and Contextual Bandits

To reiterate, a key goal of recommendation systems is to optimize for both revenue and engagement over the long run.

While embedding-based models allow to encode non-linear representations of users, items, and their interactions. However, when these representations are optimized with pointwise objectives that treat impressions or clicks as independent events, the resulting system remains fundamentally myopic.

This limitation makes sequential modeling essential. Transformers are well suited to this setting because self-attention enables the model to capture both short- and long-range dependencies across user interaction histories more effectively and efficiently than recurrent architectures [Vaswani 2017].

One of the key innovations of the Transformer architecture is its multi-head attention mechanism [Vaswani 2017]. In an advertising context, multi-head attention naturally accommodates multi-objective learning [Ma et al. 2018]. For example, one head could be tuned to prioritize short-term engagement metrics (e.g., click-through rate or CTR) by attending to recent user activities and relevant ad features, while another head could emphasize longer-term outcomes, such as mitigating ad fatigue or preserving user satisfaction [Ma et al. 2018; Zhao et al. 2019]. Each head learns a distinct set of attention weights, enabling the model to isolate and capture unique signals that serve different goals [An and Joe 2024]. The outputs from these multiple heads are then aggregated into a single representation or score, providing a unified but multi-faceted embedding of user–ad interactions [Lyu et al. 2023; Vaswani 2017]. However, determining the appropriate attention heads and their optimal weighting remains a complex challenge [Kang and McAuley 2018a; Vaswani 2017; Voita et al. 2019]. The optimal weights may vary not only across different users and content types but also over time [Zhou et al. 2019a, 2018]. Additionally, shifting business

objectives, engineering constraints, and competing optimization objectives further complicate the design process [Jannach and Abdollahpouri 2023; Mehrotra et al. 2018, 2020].

Selecting the right heads in Multi-Task Attention Models and determining their optimal weighting is not a deterministic process [Michel et al. 2019; Voita et al. 2019]. It typically requires iterative experimentation, involving both offline analysis (e.g., hyperparameter tuning, head ablation studies) and online testing (A/B experiments) [Kang and McAuley 2018b; Michel et al. 2019]. This process is particularly challenging for new users and new content, where slow adaptation can result in suboptimal personalization and early churn [Bobadilla et al. 2013; Schein et al. 2002].

To address difficulties related to policy optimization, like weight tuning and head selection, *Offline Replay* is often employed to find optimal policies in large parameter search spaces [Agarwal and Chen 2016; Agarwal et al. 2020; Levine et al. 2020]. This technique typically begins with data collection via A/B/n experiments, where different policies are randomly assigned to users [Agarwal et al. 2019; Chen et al. 2009]. Afterward, Offline Replay simulates the effects of various ad policies across different user segments and estimates the potential rewards based on logged interactions [Agarwal and Chen 2016]. In this context, "reward" refers to the performance metric we aim to maximize (i.e. clicks, likes, shares, revenue) [Wu et al. 2017]. However, in many cases, the true optimal treatment is not explicitly tested [Agarwal et al. 2019]. Here, importance sampling corrects for differences between the observed behavior policy and the target policy, improving estimation accuracy [Dudík et al. 2011; Sutton and Barto 2018]. Based on the collected data on random assignments between policies and users typically involved in Offline Replay, one can use different policy learning methods to map the optimal policies from user ids, content types, and other contextual signals for the optimal weights at any point in time [Swaminathan and Joachims 2015].

Offline replay often assumes that user preferences and item availability do not change significantly during data collection. However, in real ad systems, these factors can shift, causing offline data to become stale or biased [Agarwal et al. 2014; Levine et al. 2020]. Contextual bandits address this limitation by leveraging continuous feedback from live user interactions, preventing the policy from being locked into a static offline dataset [Li et al. 2010]. Contextual bandits can be thought of as a special case of RL, where an action selection does not influence future states [Bubeck et al. 2012; Sutton and Barto 2018]. Unlike full RL, which models long-term state transitions, contextual bandits assume that each decision is independent, optimizing for immediate rewards rather than a multi-step trajectory [Bubeck et al. 2012; Sutton and Barto 2018]. This makes them particularly well-suited for recommender systems, where ranking decisions can be treated as independent events [Afsar et al. 2022; Zhou and Brunskill 2016]. Additionally, contextual bandits naturally balance exploration and exploitation, allowing them to adapt dynamically to user behavior in real time [Li et al. 2010]. However, a practical limitation of contextual bandits is that they select actions from a fixed set defined at deployment time, such as preconfigured ad policies or treatment arms [Bietti et al. 2021]. As a result, the policy's performance is inherently constrained by the quality and granularity of the available states and actions at the time of inference [Dimitrakakis and Ortner 2018].

Although contextual bandits inherently focus on short-term rewards, they can be tuned to approximate long-term objectives by encoding historical or delayed outcomes into the feature space or the immediate reward function [McDonald et al. 2023a; Wu et al. 2017]. For instance, if an attention head emphasizing ad engagement indirectly correlates with better retention, a contextual bandit can increase its weight over time, provided that retention signals are reflected in the contextual features or the reward proxy [Yi et al. 2023]. However, full RL remains more appropriate for scenarios demanding explicit multi-step optimization, where actions taken now significantly

shape future user states [Gauci et al. 2018]. Contextual bandits thus offer a practical trade-off, providing continuous adaptation and efficient online learning without the computational complexity of fully modeling multi-step state transitions [Gauci et al. 2018; Levine et al. 2020].

Despite the practical advantages that contextual bandits offer for real-time experimentation and short-horizon reward maximization, they remain limited in scenarios where user states evolve over repeated interactions [Sutton and Barto 2018]. Modern advertising platforms often contend with extended user sessions and multiple recurring visits, demanding an approach capable of modeling multi-step feedback loops. RL naturally extends these capabilities by incorporating stateful dynamics and cumulative rewards, capturing phenomena like ad fatigue [Çelik et al. 2023] and long-term retention [Wu et al. 2017]. Recent industrial deployments underscore RL's growing importance in ads: ByteDance uses RL-based systems [Zhao et al. 2021, 2020] to balance immediate revenue with session-level engagement in short-video feeds, Meta (Facebook) has leveraged policy-gradient methods for sequencing notifications and ad placements over evolving user states [Gauci et al. 2018], and companies like Google are experimenting with RL-driven ad scheduling that accounts for shifting user contexts at scale [Zhao and Liu 2024]. These efforts demonstrate how RL can transcend the myopic focus of bandits to address long-horizon outcomes, multi-objective trade-offs, and dynamic user behaviors. In the sections that follow, we explore how these RL paradigms are integrated into large-scale advertising stacks, focusing on core algorithms, design decisions, and lessons learned from real-world implementations.

4 THE RL FORMULATION OF JOINTLY OPTIMIZING ADS AND CONTENT

4.1 Formulating Monetization-Aware Ranking as an RL Problem

4.2 Reward Design for Monetization-Aware Recommendation Systems

This subsection provides a structured synthesis of reward design in monetization-aware recommendation systems. It first discusses the properties a reward function should ideally satisfy and why reward specification is difficult in practice due to partial observability, sparse and delayed long-term outcomes, and competing objectives such as revenue, engagement, and ad fatigue. It then reviews how prior work operationalizes these objectives through proxy metrics and blended utility formulations, along with the rationale for these choices and their main limitations.

In controlled settings, relatively simple reward functions may suffice because the objective is directly specified and outcomes are immediately observable. In monetization-aware recommendation systems, by contrast, the platform must infer long-term user value from noisy behavioral traces while balancing monetization against user experience.

In RL, the reward signal provides scalar feedback used to evaluate actions and guide policy learning over time [Kaelbling et al. 1996; Sutton and Barto 2018].In principle, an effective reward signal for monetization-aware recommendation systems should satisfy at least three properties. First, it should be sufficiently informative about the user's latent preferences and intent, even though these quantities are not directly observed. Second, it should provide feedback at a timescale and frequency that makes learning feasible, even when the platform ultimately cares about sparse and delayed long-term outcomes such as retention or habit formation. Third, it should faithfully encode trade-offs among competing objectives, such as revenue, engagement, and ad fatigue, rather than overemphasize any single short-term metric in isolation.

In practice, however, satisfying these properties is difficult for at least three reasons. First, reward design is constrained by *partial observability*: the system must infer users' latent preferences and intent through a fixed set of observable signals, such as clicks, likes, shares, dwell time, or purchases, that provide only a noisy and incomplete view of the underlying user state. Second, many of the outcomes platforms ultimately care about are *sparse and delayed*: retention and longer-term

Proxy Metric	Short Description	Polarity
CTR (Click-Through Rate)	Approximates revenue potential; higher CTR often correlates with higher ad clicks [Yan et al. 2020; Zhang et al. 2018; Zhao et al. 2020].	**Positive** (higher is better)
pCTR (Predicted CTR)	Model-estimated probability that a user will click on an ad.	**Positive** (higher is better)
Predicted Revenue	Often a function of bids and pCTR to estimate expected revenue [Yan et al. 2020; Zhang et al. 2018].	**Positive** (higher is better)
Click Yield	Ratio of total clicks (ads + organic) to total impressions on a page; helps avoid optimizing CTR in isolation [Zhang et al. 2018].	**Positive** (higher is better)
Ad Impressions	Number of times an ad is displayed to users.	**Mixed** (context-dependent)

Table 4. Revenue proxy metrics and their polarity.

Proxy Metric	Short Description	Polarity
Depth	Reduction in engagement (e.g., fewer impressions per session) over time [Sagtani et al. 2023].	**Negative** (higher is worse)
Length	Decrease in time spent per session [Sagtani et al. 2023].	**Negative** (higher is worse)
Frequency	Decrease in number of sessions per week [Sagtani et al. 2023].	**Negative** (higher is worse)
Ad Load (Sessions)	Number of sessions that end (or quit) during an ad [Sagtani et al. 2023].	**Negative** (higher is worse)
Ad Load (Absolute)	Absolute ad volume across sessions and changes in that volume [Sagtani et al. 2023].	**Negative** (higher is worse)
Click Entropy	Uncertainty/diversity in ad clicks for a query; high values can indicate lower relevance [Sagtani et al. 2023].	**Negative** (higher is worse)

Table 5. Ad fatigue proxy metrics and their polarity.

satisfaction may take weeks or months to manifest, and the effect of any single intervention is often subtle, making meaningful changes difficult to detect within the short time horizons of typical experiments [McDonald et al. 2023b]. Third, reward specification is inherently *multi-objective*: recommendation engines must simultaneously account for potentially competing goals such as revenue, engagement, and ad fatigue, rather than optimize a single scalar outcome in isolation [Sagtani et al. 2024; Yan et al. 2020; Zhang et al. 2018].

These challenges make reward design inherently difficult, since the platform's true objectives are not directly observed but must instead be approximated through sparse and imperfect proxies. Click-through rate (CTR) and ad impressions serve as approximate rewards for revenue, while session duration, saves, and shares act as proxies for longer-term engagement [Wang et al. 2022]. However, these simpler proxies can fail to capture the system's true long-term objectives, occasionally leading to suboptimal policies [Mazoure et al. 2021]. A high CTR may reflect short-term engagement, yet it can ultimately induce ad fatigue or frustrate users if the ads are misaligned with their actual interests [Hohnhold et al. 2015b; Sagtani et al. 2024]. Tables 4, 5, and 6 summarize commonly used proxies in the industry for revenue, ad fatigue, and engagement.

While Tables 4, 5, and 6 summarize a wide range of proxy metrics for revenue, ad fatigue, and engagement, selecting any individual proxy requires care. Many proxies are attractive precisely

Proxy Metric	Short Description	Polarity
Session Duration	Tracks how long users stay engaged per session (proxy for longer-term engagement).	**Positive** (higher is better)
30-day Total Sessions	Aggregated count of user sessions over a 30-day window, proxy for habit or stickiness [Xu et al. 2023].	**Positive** (higher is better)
Likes / Adds to Library	High-intent actions indicating deeper user satisfaction and long-term retention [McDonald et al. 2023b].	**Positive** (higher is better)
Shares	High-intent action reflecting strong engagement.	**Positive** (higher is better)
High-quality Consumption	Consumption of content with high engagement (e.g., full video views, long reads) [Wang et al. 2022].	**Positive** (higher is better)
User Return Rate (URR)	Fraction of users who revisit the platform after a fixed interval [Wu et al. 2017].	**Positive** (higher is better)
Relative Watch Time	Percentage of a video watched, normalized by video length [Wu et al. 2018].	**Positive** (higher is better)
Dwell Time	Actual time spent viewing/reading an item, capturing depth of engagement [Yi et al. 2014].	**Mixed** (threshold-dependent: < 5s negative; ≥ 10s positive)
Ratio-based Diversity	Proportion of unique topic clusters in consumption history [Wang et al. 2022].	**Mixed** (context-dependent)
Repeated Consumption	How often users revisit similar content, reflecting satisfaction [Wang et al. 2022].	**Mixed** (context-dependent)
Distribution-based Diversity	Entropy-based spread of consumed topics [Wang et al. 2022].	**Mixed** (context-dependent)
Short-view Rate (SVR)	Fraction of impressions where watch time is below a low threshold (e.g., < 5s) [Pan et al. 2023].	**Negative** (higher is worse)
Early-skip Rate	Fraction of videos skipped or swiped away within the first few seconds (e.g., < 2 to 3s) [Pan et al. 2023].	**Negative** (higher is worse)
Skip / Not-click Rate	Fraction of impressions with explicit negative feedback signals (e.g., skips or non-clicks), indicating low relevance [Zhao et al. 2018b].	**Negative** (higher is worse)
Non-completion Rate	Fraction of plays that end before a minimum completion threshold (e.g., $< 20\%$ watched) [Pan et al. 2023].	**Negative** (higher is worse)
Time to Revisit	Interval between consecutive visits; shorter intervals suggest higher retention [Wang et al. 2022].	**Negative** (higher is worse)

Table 6. Engagement proxy metrics and their polarity.

because they provide dense, behaviorally grounded feedback that is easy to log and optimize. Click-based metrics are attractive because they are dense and immediately observable, but prior work cautions that they can overstate true user value: high CTR need not imply high ad quality, and repeated exposure to low-quality or poorly matched ads can reduce long-term user satisfaction and future responsiveness to ads [Hohnhold et al. 2015a].

. Even within the same signal family, semantics can vary: short clicks are often interpreted as implicit negative feedback, whereas longer clicks or post-click dwell can be more indicative of genuine value. Similar tensions arise for other objectives. Diversity-related signals, for instance, may be desirable when they reflect broadening of user interests over time, yet high click entropy within a session can also indicate confusion, low relevance, or friction. Taken individually, these proxies can be valuable signals; taken together, however, they reveal that reward design is not simply a

matter of choosing "good" metrics, but of reconciling partially informative, context-dependent, and sometimes conflicting measurements of user value. This is precisely what motivates the use of composite utility functions that blend multiple proxy signals rather than relying on any single one in isolation.

A **canonical utility function** at the policy layer to *blend* ads and organic content can be represented as follows [Carrion et al. 2021; Sagtani et al. 2024; Yan et al. 2020]:

$$U(x) = \alpha \times \text{Revenue} + \beta \times \text{Engagement} - \gamma \times \text{Ad Fatigue} \tag{1}$$

where α, β, and γ are weights that determine the relative importance of each factor. To compute the utilities of "Revenue", "Engagement", and "Ad Fatigue", typically many signals feed into each component. These signals include, but are not limited to, the ones shown in Tables 4, 5, and 6. In a bandit or RL context, $U(x)$ serves as the reward signal guiding policy updates. The process of tuning the weights of α, β, and γ is often done through experimentation, as the right balance between engagement and revenue can shift depending on the context. For instance, ByteDance uses a multi-objective reward framework in their recommendation systems, where different reward functions are designed for organic content and ads, combining them to guide the overall recommendation strategy [Zhao et al. 2020].

In some cases, a very simple unified function can suffice over a complex utility function. Researchers at Alibaba introduced the concept of Click Yield, which is defined as the ratio of the total number of clicks on all items (ads + organic results) to the total number of impressions on a page [Zhang et al. 2018]. Click Yield provides a holistic evaluation of page performance, accounting for the interactive effects between ads and organic content, and helping to mitigate the bias that arises from examining CTR in isolation. Their optimization task maximizes total revenue while ensuring that the Click Yield does not fall below a certain threshold (T) [Zhang et al. 2018]. A comparison of utility function formulations is shown in Table 7.

Company	Paper	Organic Content Reward/Utility	Ads Reward/Utility	Blended Reward/Utility
TikTok	[Zhao et al. 2021]	User Experience (r_t^{ex}): Measures the negative impact of displaying an ad on the user's experience, with a binary outcome. A positive reward (1) is given if the user continues browsing after seeing the ad, while a negative reward (-1) is given if the user leaves the platform after seeing the ad.	Ad Income (r_t^{ad}): Represents the immediate revenue generated from displaying an ad. It provides a positive value if an ad is displayed, contributing to maximizing advertising income.	$r_t(s_t, a_t) = r_t^{ad} + \alpha \cdot r_t^{ex}$ Here, α is a hyperparameter that controls the trade-off between maximizing ad revenue and minimizing negative user experience.
TikTok	[Zhao et al. 2020]	$R_{\text{rec}}(s_t, a_t)$ Engagement metric (e.g., dwell time, clicks, purchases)	$R_{\text{ad}}(s_t, a_t) = \begin{cases} 1 & \text{if user stays after ad} \\ 0 & \text{if user leaves after ad} \end{cases}$	$R_{\text{total}}(s_t, a_t) = \alpha \cdot R_{\text{rec}}(s_t, a_t) + \beta \cdot R_{\text{ad}}(s_t, a_t)$
Alibaba	[Zhang et al. 2018]	Click Yield (CY): Measures the overall performance of the entire page, considering both ads and organic content. It is defined as the ratio of total clicks on all items (both ads and organic) over the total number of impressions. $CY_{ij} = \frac{\sum_{n=1}^{N} Click(i,j,n)}{Imp(i,j)}$ Where $Click(i,j,n)$ is the click for item n in list j, and $Imp(i,j)$ is the number of impressions.	Revenue (REV): For sponsored content, revenue is computed based on the click-through rate (CTR) and cost per click (CPC). $REV_{ij} = \sum_{n=1}^{N} CTR(i,j,n) \times CPC(i,j,n)$ The CPC is determined by the auction mechanism (usually Generalized Second Price Auction).	The system optimizes the trade-off between revenue (REV) and Click Yield (CY). The optimization problem is defined as: $\max \sum_{i,j} REV_{ij} \cdot x_{ij}$ Subject to the constraint: $\frac{\sum_{i,j} CY_{ij} \cdot x_{ij}}{\sum_{i,j} x_{ij}} \geq T$ Where T is a threshold for the average Click Yield, ensuring user satisfaction.
LinkedIn	[Yan et al. 2020]	Expected Engagement Utility: Measures user actions like clicks, comments, shares, or time spent on organic content. The goal is to maximize engagement with organic content.	Expected Revenue Utility (for ads): Evaluates the revenue generated from ads, typically calculated as the product of the bid amount and the predicted click-through rate (pCTR).	The utility function for ads and organic content is formulated as: $\max \sum_i x_i r_i - \frac{w}{2}\|x\|^2$ Subject to the engagement constraint: $\sum_i x_i u_{ai} + (1 - x_i) u_{oi} \geq C$ Where: • x_i is a decision variable determining whether to place an ad in position i, • r_i represents the revenue utility for ad i, • u_{ai} and u_{oi} represent the engagement utilities for ads and organic content, respectively, • C is the engagement threshold. The shadow bid α acts as a conversion factor to blend the two objectives by equating engagement utility and revenue in a comparable manner.

Table 7. Comparison of Utility/Reward Formulation Approaches across Different Companies

4.3 Action Space Representation

In RL, an **action** $a \in \mathcal{A}$ refers to a decision or intervention taken by the agent based on its current state to influence the environment [Sutton and Barto 2018]. The action space $\mathcal{A}$ represents the complete set of all possible actions that the agent can choose from at any given decision point

[Sutton and Barto 2018]. By contrast, in an *advertisement recommendation* setting, the action space $\mathcal{A}$ might consist of all possible combinations of ads, their rankings, placements, and formats [Ie et al. 2019b]. The objective of this set of actions in ad recommendation is to deliver the right set of ads at the right time in order to maximize platform revenue, while maintaining or improving user engagement and satisfaction [Zhao et al. 2020]. Although this action space is discrete and countable, the number of possible combinations grows exponentially with the number of candidate ads and available slots. As a result, it becomes computationally infeasible to enumerate or evaluate each action individually, and must instead be treated using methods designed for high-dimensional discrete spaces [Ie et al. 2019a].

An ideal action space should be *expressive* enough to capture all relevant degrees of freedom the agent needs to optimize its behavior, but it should avoid unnecessary complexity [Liu et al. 2025; Zhu et al. 2024]. The action space described above in the FrozenLake example satisfies these criteria, as the four cardinal movements (*up, down, left,* and *right*) are enough to navigate to any state in the grid while remaining **orthogonal**, thus providing both sufficient coverage and simplicity [Gym Documentation Team 2024]. In contrast, the ad recommendation action space is inherently high-dimensional and entangled [Ie et al. 2019a]. The agent must decide *what* to show (a discrete choice over candidate ads), *where* to place it (which may be modeled as a discrete slot or a continuous position), and in *what order* (often determined by continuous relevance scores), often under tight latency constraints and user personalization requirements [Zhao et al. 2020]. The action space in ad recommendation systems must be expressive enough to encompass all relevant choices for optimal ad delivery while ensuring no combinatorial explosion in complexity [Ie et al. 2019b]. Moreover, ensuring that each dimension remains sufficiently *orthogonal* can be nontrivial, given potential interactions between ad content, positioning, and user context [Ie et al. 2019c]. If the action space is too coarse, the system risks underfitting, serving generic policies that fail to personalize effectively or miss monetization opportunities [Ie et al. 2019b]. On the other hand, if it is too fine, the system may overfit to noise, struggle to generalize, or incur prohibitive computational costs [Ie et al. 2019b]. Practical systems often resort to action abstraction or parameterization (e.g., slate-based policies, ad load knobs, or policy buckets) to reduce complexity at the cost of optimality [Hohnhold et al. 2015b; Ie et al. 2019c; Sagtani et al. 2024].

The independence of these action dimensions facilitates more modular policy learning [Sharma et al. 2017; Tavakoli et al. 2018]. By allowing a degree of independence for each action dimension, the agent achieves modular policy learning that scales linearly (instead of exponentially) with the number of action degrees-of-freedom [Tavakoli et al. 2018]. In addition, actions should ideally correspond to semantically meaningful choices to facilitate policy interpretability and validation [Glanois et al. 2024; Puiutta and Veith 2020]. In the case of the FrozenLake problem, the action space is intuitive and human-interpretable, making it straightforward to validate learned policies [Brockman et al. 2016; Sutton and Barto 2018]. For instance, an RL practitioner can manually inspect and reason about the learned policy (e.g. "the agent plans to move *down* then *right* to reach the goal") to verify that each action choice aligns with domain logic [Brockman et al. 2016]. While the action space in ad recommendation systems is far more complex, structuring it around semantically meaningful levers (e.g., "increase ad load for high-intent users" or "down-rank low-quality creatives") enables engineers and product teams to reason about the system's behavior [Dulac-Arnold et al. 2021; Glanois et al. 2024]. If a learned policy increases ad load for a particular user segment, analysts can trace this decision back to upstream signals, such as historical engagement patterns or commercial intent scores [Dulac-Arnold et al. 2021; Gauci et al. 2018]. Policies composed of interpretable levers enable more transparent debugging, offline validation, and policy review, which is crucial for ensuring that the deployed strategy aligns with business

intuition, avoids learning spurious correlations, and mitigates the risk of unintended outcomes [Dulac-Arnold et al. 2021; Glanois et al. 2024; Puiutta and Veith 2020; Verma et al. 2018].

Moreover, an action space should be consistent across episodes [Puterman 2014]. That is, taking the same action in the same state should yield statistically predictable outcomes [Puterman 2014]. The state space in the FrozenLake is stochastic [Gym Documentation Team 2024], meaning that taking the same action does not always result in the same next state. For example, selecting the action "move right" might result in the agent moving right, up, or down, as governed by the transition probabilities defined by $P(s' \mid s, a)$ [Gym Documentation Team 2024]. Despite the stochasticity in the environment, the outcomes observed should converge to the expected values described by the transition probability distribution over a sufficiently large number of episodes [Puterman 2014]. A similar principle applies in ad recommendation. Although user behavior and auction dynamics introduce stochasticity, taking the same ad-serving action, such as presenting a specific slate of ads to a user with a given context, should yield statistically predictable engagement or monetization outcomes. That is, over many episodes, the distribution of observed rewards (e.g., clicks, conversions, revenue) should converge to the expected value associated with the given state-action pair [McMahan et al. 2013; Zhao et al. 2018a]. This consistency is foundational for policy learning, as it ensures that repeated exposure to similar decision contexts enables accurate estimation of long-term value, despite noisy observations [Sutton and Barto 2018].

The action space can be thought of as levers that can be used to tune ad policy. Typical **levers in treatments in ad policies** include the following elements [Koutsopoulos and Spentzouris 2016]:

- **Ad Load.** Also referred to as the spacing ads or frequency of ads is also a key component in inducing ad load fatigue. Placing ads too closely together may lead to user fatigue and reduced click probabilities [Abrams and Vee 2007; Zhao et al. 2020].
- **Ad Quality.** This refers to intrinsic features of the ad, such as the design format, product type, accompanying text, and the quality of the landing page, all of which directly influence user engagement. [Nielsen 2017; Yan et al. 2009].
- **Ad Placement.** Ads placed earlier in the feed are more likely to be clicked, though their relevance to preceding content also plays a significant role [Saha et al. 2021; Yuan et al. 2020; Zhao et al. 2020].
- **Ad Relevance.** Ads that align more closely with user preferences and user intent are more likely to drive engagement. For example, an ad on a hotel might have a higher probability if shown adjacent to results of search queries related to vacations [Chakrabarti et al. 2008; Zhou et al. 2018].

4.4 State Space Representation

In RL, a *state* $s \in \mathcal{S}$ represents all the relevant information from past agent-environment interactions necessary for the agent to make optimal decisions to maximize the expected cumulative reward [Sutton and Barto 2018]. Ideally, the state space is Markovian, meaning the future is conditionally independent of the past given the current state [Russell and Norvig 2016; Sutton and Barto 2018]. If the agent's policy or value estimates rely on omitted or unobserved information from the past, the state representation violates the Markov property and the performance of the learned policy may degrade as a result [Russell and Norvig 2016; Sutton and Barto 2018]. In the FrozenLake example, the state space consists of the agent's current grid position as the future outcomes of any action depend only on this position, and not on how the agent arrived there [Gym

Documentation Team 2024]. In this example, because future transitions depend only on the current state and action, and not on prior history, the FrozenLake Environment satisfies the *Markov Property* [Gym Documentation Team 2024; Sutton and Barto 2018].

In contrast, the environments where ad recommendation systems operate are often partially observable, high-dimensional, and non-stationary [Dulac-Arnold et al. 2021; Russell and Norvig 2016]. To construct a state representation that reasonably approximates the Markov property in the face of partial observability and complex user dynamics, ad recommendation systems often combine tabular features with learned embeddings [Afsar et al. 2022; Kaelbling et al. 1998]. The tabular signals tend to include static attributes (e.g. demographics), contextual metadata (e.g. device type, time of the day), and behavioral history (e.g. recent clicks, impressions, or dwell time) [Cheng et al. 2016; Covington et al. 2016]. An embedding is a learned, dense vector that encodes complex or high-cardinality inputs, such as user profiles, ads, or behavioral sequences, into a lower-dimensional space [Cheng et al. 2016]. Compared to static tabular features like frequency counts or recency buckets, embeddings capture richer structure, including sequence effects and latent similarity. This makes them more expressive and task-relevant, enabling ad recommendation systems to personalize content and ads more effectively [Cheng et al. 2016; Covington et al. 2016].

Embeddings may consist of two components: an offline (stationary) component and an online (non-stationary) component [Zhao et al. 2023]. **Offline embeddings** represent **stationary or slow-changing attributes**, such as a user's demographics (e.g., gender, country) or long-term preferences, and fixed item properties (e.g., brand or product category) [Koren et al. 2009; Quadrana et al. 2018]. These embeddings are usually computed in batch mode due to their **higher computational complexity, larger size, and relatively static nature** [Liu et al. 2022; Naumov et al. 2019]. **Online embeddings**, conversely, address non-stationary and rapidly evolving aspects such as recent user behaviors, trending content, or immediate interests [Zhou et al. 2019b, 2018]. These embeddings are **generally smaller, computed in near-real-time to meet latency requirements** [Liu et al. 2022]. This hybrid approach combines stable and dynamic embeddings, balancing relevance with computational efficiency [Cheng et al. 2016; Covington et al. 2016]. Furthermore, this allows the recommendation system to remain stable under distribution shifts, which is of paramount importance, especially in environments with rapidly evolving user preferences, content inventories, and behavioral patterns [Gama et al. 2014; Sutton and Barto 2018].

To effectively integrate diverse embeddings (user, item, interaction) and their offline and online components, embeddings of varying dimensions are typically concatenated and then projected via learned linear layers into a shared dimension [Covington et al. 2016; Grbovic and Cheng 2018; Pancha et al. 2022]. Generally, zero-padding or truncation is avoided, as these methods can introduce noise or cause information loss [Xia et al. 2023].

User embeddings are learned vector representations designed to capture user preferences and behaviors, enabling ad recommendation systems to personalize content effectively at scale [Covington et al. 2016; He et al. 2017]. These embeddings compress complex, high-dimensional user data, such as past interactions and demographic information, into compact vectors utilized by recommendation models [Covington et al. 2016; Koren et al. 2009; Naumov et al. 2019]. User embeddings can be learned using a combination of supervised, unsupervised, and self-supervised approaches [Zhao et al. 2021]. In the case of SL, embeddings are optimized to predict explicit labels such as clicks, conversions, or purchases [Covington et al. 2016; Zhou et al. 2018]. To exploit information in more implicit signals, self-supervised methods can be used with objectives such as next item interaction or item co-occurrence [Mikolov et al. 2013; Yuan et al. 2020]. Many practical implementations tend to combine both supervised and self-supervised methods [Xia et al. 2023; Zhao et al. 2021]. For example, embeddings may initially be learned through self-supervised tasks (e.g., predicting next clicks) and then fine-tuned on supervised tasks (CTR, conversions). A

well-documented example is that of YouTube, where despite the availability of explicit feedback mechanisms (thumbs up/down, in-product surveys, etc.), they turn to implicit signals, where a user completing a video is a positive example, given that the relative prevalence of implicit signals is much higher than explicit signals, allowing the models to learn more effectively [Covington et al. 2016].

Item embeddings are designed to capture intrinsic item characteristics [Koren et al. 2009]. These embeddings compress complex, high-dimensional item data, such as product descriptions, categories, brands, and historical interaction patterns [Cheng et al. 2016; Koren et al. 2009]. Similar to user embeddings, item embeddings can be learned through supervised, unsupervised, and self-supervised approaches [Barkan and Koenigstein 2016; Zhao et al. 2023]. In SL scenarios, embeddings are optimized to predict explicit labels, such as clicks, conversions, or purchases associated with the items [Covington et al. 2016; Naumov et al. 2019]. To leverage more implicit information, self-supervised methods may focus on objectives like predicting co-occurrence patterns or sequential interactions (e.g., item-to-item relationships) [Barkan and Koenigstein 2016; Sun et al. 2019].

While user and item embeddings provide essential representations of individual entities, it is through interaction embeddings, which capture the joint dynamics between users and items, that the system truly learns user preferences and can surface content that is contextually relevant and personalized [Koren et al. 2009]. Being able to **effectively represent user-item interactions** is at the core of personalization for recommendation systems [Linden et al. 2003]. Collaborative Filtering (CF) was one of the earliest popular methods to represent user-item interactions [Bobadilla et al. 2013; Linden et al. 2003]. CF predicted user preferences by analyzing explicit interactions (e.g., ratings) or implicit behavioral signals (e.g., clicks, views), using metrics like cosine similarity or Pearson correlation [Bobadilla et al. 2013]. However, CF struggled with sparse datasets, as many user-item pairs lacked interactions, limiting recommendation accuracy [Bobadilla et al. 2013]. Additionally, CF's direct interaction-based approach limited its ability to capture deeper latent or implicit relationships between users and items [Koren 2008].

Matrix Factorization (MF), popularized during the Netflix Prize competition (2006–2009), addressed some of CF's key limitations [Bennett and Lanning 2007]. The classic approach is to factorize the (sparse) interaction matrix into the product of a user-factor matrix and an item-factor matrix [Koren et al. 2009]. MF learns these latent (hidden) user and item factors by optimizing embeddings to reconstruct observed interactions, enabling it to capture deeper relational structures that CF missed [Koren et al. 2009]. MF also significantly improved storage efficiency [Koren et al. 2009]. For instance, a CF matrix with 1 million users and 1 million items would contain 1 trillion (mostly sparse) entries. In contrast, MF represents these interactions using two dense embedding matrices totaling approximately 128 million entries (assuming an embedding dimension of k=64) [Koren et al. 2009].

As companies began logging large-scale user interaction data in the late 2000s and early 2010s, Amazon, Spotify, and Netflix turned to Matrix Factorization (MF) as a simple yet effective way to harness that data for scalable personalization [Gomez-Uribe and Hunt 2015; Linden et al. 2003]. However, while MF improved predictions for existing users/items with limited interactions, it did not solve all limitations of CF [Koren et al. 2009]. Particularly, MF still struggled with the cold-start problem, as completely new users or items without interaction history cannot be effectively represented in the embedding space [Koren et al. 2009]. Additionally, MF assumes linear interactions between latent factors, limiting its ability to capture more complex, nonlinear user behaviors [He et al. 2017]. Neural embeddings emerged in the 2010s to address some of these limitations [He et al. 2017]. Unlike CF and MF, neural embeddings naturally capture complex, nonlinear interactions between users and items [He et al. 2017]. They also effectively incorporate content-based and

contextual metadata, significantly improving the handling of cold-start issues [Cheng et al. 2016; Naumov et al. 2019; Van den Oord et al. 2013]. Two-tower models, notably introduced by Google for YouTube's recommendation systems, leverage deep neural networks for scalable candidate retrieval and generation [Covington et al. 2016]. A two-tower model consists of two distinct neural network components: one tower encodes user-specific features, and the other one encodes item-specific features, projecting them into a shared latent embedding space [Covington et al. 2016]. This shared embedding space enables efficient similarity computation, facilitating large-scale real-time recommendations [Covington et al. 2016]. Today, companies like YouTube use two-tower models for tasks such as next-video prediction, and Meta uses them for personalized ad ranking [Covington et al. 2016; He et al. 2014; Naumov et al. 2019].

Following the widespread adoption of two-tower neural networks, the need to explicitly capture temporal dynamics emerged, motivating the rise of sequence-aware embeddings [Quadrana et al. 2018]. Users' preferences evolve continuously, and earlier methods struggled to effectively capture these dynamics [Gama et al. 2014]. Earlier embedding methods, such as Matrix Factorization and two-tower models, primarily represented user-item relationships statically, without adequately capturing evolving user preferences or the order of interactions [Covington et al. 2016; Koren et al. 2009]. Sequence-aware embeddings thus emerged as an important advancement [Quadrana et al. 2018]. Initially, RNNs, particularly GRUs, were introduced to model sequential user behaviors [Kang and McAuley 2018b]. GRUs could effectively capture short-term temporal dependencies by processing sequences step-by-step, making them well-suited for real-time recommendation tasks [Kang and McAuley 2018b; Zhao et al. 2021]. However, these recurrent architectures faced several key limitations: they were prone to the vanishing gradient problem, where information from earlier interactions diminished during training, making it challenging to capture long-range dependencies [Bengio et al. 1994]. They also exhibited a recency bias, disproportionately emphasizing recent interactions and overlooking longer-term user interests [Pascanu et al. 2013]. Moreover, their sequential nature resulted in slow training times, limiting their scalability [Shim and Sung 2022]. To address these limitations, Transformers, originally introduced by Vaswani et al. in 2017, became widely adopted for recommendation tasks in the late 2010s and early 2020s [Vaswani 2017]. Transformers leveraged self-attention mechanisms, allowing the model to simultaneously process all interactions in a sequence and dynamically attend to the most informative interactions—such as pivotal clicks, conversions, or moments of high engagement [Vaswani 2017]. This adaptive attention mechanism effectively mitigated the vanishing gradient problem, robustly captured both short-term and long-range dependencies, and reduced recency bias by emphasizing interactions based on relevance rather than mere chronological proximity [Kang and McAuley 2018b; Vaswani 2017].

Transformers enable parallel processing over sequence elements, significantly accelerating training and making them well-suited for large-scale recommendation systems with extensive historical data [Kang and McAuley 2018b; Vaswani 2017]. Industry models like Pinterest's PinnerFormer (2020) and ByteDance's DEAR (2021) have leveraged this capability to effectively model evolving user behavior and serve contextually relevant recommendations [Pancha et al. 2022; Zhao et al. 2021].

4.5 Policy Learning

In the context of **ad policy optimization**, the policy $\pi(a|s)$ defines how the system selects ads a to display to a user in a particular state s, using levers such as ad load, placement, and price thresholds, as detailed in Section 4.3 [Ie et al. 2019c; Zhang et al. 2018; Zhao et al. 2021]. The state s may include the user's profile, browsing history, session context, and inferred commercial intent, as discussed in Section 4.4 [Pancha et al. 2022; Xia et al. 2023; Zhou et al. 2019a]. The policy can

be deterministic (always show the same ad for a given state) or stochastic (show ads based on probabilities) [Levine et al. 2020; Sutton and Barto 2018]. In advanced recommendation systems, ad policies are typically stochastic, whereas in older recommendation systems, policies might be deterministic (i.e. every X impressions of organic items show ad Y from list Z) [Gauci et al. 2018; Zhao et al. 2021]. **Stochastic policies** are desirable because they **naturally support exploration**, allowing the system to sample from a range of actions rather than always selecting the historically best-performing one [Russo et al. 2018; Sutton and Barto 2018]. This **helps avoid local optima** and uncover ad-user pairings that may lead to better long-term outcomes [Mazoure et al. 2021; Zou et al. 2019]. Moreover, stochastic policies provide a practical mechanism for managing uncertainty and partial information [Dulac-Arnold et al. 2021; Levine et al. 2020]. When it is unclear which action best serves all objectives, such as monetization and long-term engagement, randomizing over plausible actions allows the system to spread risk and continue learning [Jannach and Abdollahpouri 2023; Mehrotra et al. 2020]. While **deterministic policies** can in principle encode complex tradeoffs, they may hard-code assumptions that **do not generalize well or require re-tuning as user behavior shifts** [Dulac-Arnold et al. 2021]. Stochastic policies, by contrast, can naturally adapt via probabilistic weighting as more data becomes available [Gauci et al. 2018; Levine et al. 2020].

Learning a policy involves a series of critical design decisions that must account for both algorithmic trade-offs and real-world constraints. These include whether to learn from historical data in an **offline** setting or through direct interaction with the environment via **online learning**; whether to rely on **value-based methods** or **policy-based** optimization; whether to incorporate an explicit model of the environment or adopt a model-free approach; and whether to use tabular methods or generalize through function approximation. In addition, selecting an appropriate exploration strategy is essential for balancing the trade-off between learning new behaviors and exploiting known high-reward actions. This section provides guidance on how to navigate these decisions, using the FrozenLake environment as a conceptual reference point and highlighting real-world implementations from large-scale recommendation systems.

4.5.1 Offline vs Online Policy Learning. Depending on whether the agent learns by training on historical data or by interacting with the environment in real time, policy learning can be categorized as either offline or online [Sutton and Barto 2018]. In offline policy learning, models are trained using logged data from past user interactions, without actively modifying the live system [Levine et al. 2020]. This is often the safest and most scalable approach in large platforms, as it avoids the risk of deploying untested policies [Levine et al. 2020]. Offline methods are particularly useful during the early stages of development when real-time exploration is expensive [Gauci et al. 2018; Levine et al. 2020; Mehrotra et al. 2020].

Monte Carlo (MC) methods estimate the expected return of a policy by averaging cumulative rewards across complete episodes of logged interactions [Levine et al. 2020; Sutton and Barto 2018]. Because they require observing full trajectories before updates, MC methods are particularly well-suited to settings where rewards are delayed and live exploration is costly [Levine et al. 2020; Sutton and Barto 2018]. This makes them a natural fit for early-stage offline learning scenarios, where the focus is on extracting long-term value insights from existing data without interacting with the live environment [Levine et al. 2020; Sutton and Barto 2018]. However, Monte Carlo methods have notable limitations [Sutton and Barto 2018]. Waiting for full episodes to complete can make them highly sample-inefficient, particularly in environments where episodes are long or rewards are sparse [Levine et al. 2020]. This inefficiency can delay policy improvements and slow adaptation to changes [Dulac-Arnold et al. 2019]. Furthermore, MC estimates often suffer from high variance, especially in noisy or stochastic environments, which can destabilize learning

unless techniques like reward normalization, batch averaging, or variance reduction are applied [Sutton and Barto 2018]. Monte Carlo approaches also assume that logged episodes are representative of the environment; if the logging policy is narrow or biased, importance sampling corrections become necessary to avoid skewed estimates [Swaminathan and Joachims 2015]. Due to these limitations, Monte Carlo methods are typically not used in high-frequency online systems where quick updates are critical [Dulac-Arnold et al. 2019]. As an illustrative example of offline learning, in the context of the FrozenLake environment, offline policy learning refers to the process of learning a policy using a log of past episodes, i.e. sequences of (state, action, next state, reward) generated by a prior policy. The learner must rely entirely on the logged data to reason about optimal behavior in navigating from the start state (S) to the goal (G), while avoiding holes (H) and dealing with slippery transitions.

Logged data only contains actions chosen by the historical policy, which means there is no direct visibility into what would have happened if alternative actions had been taken in the same context [Swaminathan and Joachims 2015]. This lack of counterfactual information makes it difficult to evaluate or iterate on new policies [Dudík et al. 2011]. However, methods such as inverse propensity scoring (IPS), which re-weights logged rewards to approximate outcomes under a different policy, and doubly robust estimators, which combine IPS with model-based value predictions, can help address this challenge [Dudík et al. 2011; Levine et al. 2020; Swaminathan and Joachims 2015]. These approaches enable limited forms of counterfactual reasoning, especially when the new policy does not diverge too far from the one that generated the data [Dudík et al. 2011; Swaminathan and Joachims 2015].

A simple example of counterfactual estimation being viable in the FrozenLake environment can be illustrated as follows: suppose the agent has never made a left turn from state 4. While we may not have logged data on this specific action at this specific grid location, a function approximator—such as a neural network trained over state-action pairs—might still be able to infer the expected outcome. This is because the model could have learned to generalize from similar patterns observed in nearby states, allowing it to estimate the return of an unobserved action in a plausible way [Dudík et al. 2011; Levine et al. 2020].

To overcome the constraints of limited exploration in offline settings, online policy learning enables agents to refine their policies by interacting with the environment and learning from new experiences [Dimitrakakis and Ortner 2018]. The agent is able to explore different actions, observe the consequences in real time, gather feedback, and update its behavior accordingly. This ability to learn directly through interaction with the environment makes online learning more flexible and adaptive, although it also introduces greater risk [Dulac-Arnold et al. 2019].

To mitigate risks and costs of online exploration, offline is almost always used first [Levine et al. 2020]. Online is introduced gradually via controlled bandit loops, feature gates, or soft rollouts. This hybrid approach of offline-then-online policy learning is commonly seen in industry. In their offline RL system, YouTube trained an actor-critic model entirely on logged user interaction data, using Monte Carlo returns to estimate long-term value, and only deployed policies to production after thorough offline evaluation [Chen et al. 2022]. At Meta, the Horizon platform first trains policies offline using counterfactual estimators like Doubly Robust evaluation, and then deploys them through gated rollouts, which refer to the controlled and staged release of new policies to a limited portion of traffic in order to minimize risk during online learning [Gauci et al. 2018]. Spotify similarly adopts a cautious approach by first optimizing contextual bandit policies offline, and then introducing online learning through incremental updates and careful exploration strategies [Mehrotra et al. 2020].

4.5.2 Value-based vs Policy-based Learning. RL algorithms can be divided into value-based and policy-based methods based on how they learn to make decisions to maximize cumulative rewards [Russell and Norvig 2016; Sutton and Barto 2018]. Value-based methods first estimate the value of states or state-action pairs and then derive a policy from these estimates, whereas policy-based methods learn the policy directly by optimizing expected cumulative reward [Schulman et al. 2017; Sutton and Barto 2018]. In other words, value-based methods focus on learning the utility of states or actions and derive the optimal behavior indirectly by acting greedily with respect to this learned value [Sutton and Barto 2018]. Policy-based methods, in contrast, directly optimize the policy without estimating value functions [Schulman et al. 2017; Sutton and Barto 2018].

Because value-based methods rely on greedy action selection to determine the next best action, they require comparing the estimated value of all possible actions at each decision point (e.g. each state) [Dulac-Arnold et al. 2019; Sutton and Barto 2018]. As the number of available actions grows, and as the agent encounters more states during interaction, this exhaustive comparison becomes computationally intractable [Dulac-Arnold et al. 2019]. For this reason, value-based methods are particularly well-suited for environments with discrete or low-dimensional action spaces, where enumerating and comparing action values remains computationally feasible [Dulac-Arnold et al. 2019; Sutton and Barto 2018]. Common algorithms used in value-based learning include Q-learning, SARSA, and Deep Q-Networks (DQN) [Mnih et al. 2015; Sutton and Barto 2018].

In Policy-based learning, the agent tries out different strategies and tweaks their behavior directly based on what works well overall, rather than scoring every possible move [Russell and Norvig 2016; Sutton and Barto 2018]. Formally, the goal is to learn a parameterized policy $\pi_\theta(a \mid s)$, where θ denotes the parameters of the policy [Schulman et al. 2017; Sutton and Barto 2018]. In practice, especially in modern industrial applications, these parameters are often the weights of a neural network, though other function approximators are possible [Levine et al. 2020; Mnih et al. 2016; Sutton and Barto 2018]. Neural network-based policies have become the norm in large-scale recommendation systems due to their expressiveness and scalability, making them particularly well-suited for environments with complex and high-dimensional state spaces [Chen et al. 2022; Gauci et al. 2018; Lin et al. 2023]. Formally, the objective function to be maximized is the expected cumulative reward under the policy:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{\infty} \gamma^t R_{t+1} \right] \tag{2}$$

where τ denotes a trajectory: a sequence of states and actions generated by following the policy π_θ [Levine et al. 2020; Sutton and Barto 2018]. To optimize this objective, the policy gradient theorem provides a way to compute the gradient of the expected return with respect to the policy parameters, given by:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta} \left[\nabla_\theta \log \pi_\theta(a \mid s) \cdot Q^{\pi_\theta}(s, a) \right] \tag{3}$$

where $Q^{\pi_\theta}(s, a)$ is the action-value function under the current policy [Sutton and Barto 2018; Williams 1992]. Since the goal is to maximize return, gradient ascent, not descent, is used [Sutton and Barto 2018; Williams 1992]. Common policy-based RL algorithms include REINFORCE, Proximal Policy Optimization (PPO), Trust Region Policy Optimization (TRPO), and various Actor-Critic methods, which combine both policy and value learning in a hybrid architecture [Schulman et al. 2015, 2017; Sutton and Barto 2018; Williams 1992].

In theory, either value-based or policy-based learning is sufficient to solve an RL problem [Kaelbling et al. 1996; Sutton and Barto 2018]. However, policy-based methods tend to be more suitable for ad policy optimization for a number of reasons [Chen et al. 2022; Gauci et al. 2018]. First,

policy-based methods are preferred in scenarios where stochastic policies are desired because they explicitly parameterize the policy as a probability distribution over actions, rather than implicitly choosing a single best action [Sutton and Barto 2018; Williams 1992]. What policy-based methods learn are the stochastic policy $\pi_\theta(a \mid s)$ directly, whereby the output is itself a probability distribution (e.g. softmax over action logits, Gaussian in continuous space) [Schulman et al. 2015; Sutton and Barto 2018]. Moreover, policy-based methods are also desirable when careful tuning of exploration-exploitation is required, as they can learn when and how to be random, whereas value-based methods cannot [Schulman et al. 2015; Sutton and Barto 2018; Williams 1992]. In contrast, because value-based methods learn the action-value function $Q(s, a)$ and then derive the deterministic policy $\arg\max_a Q(s, a)$, value-based methods cannot directly represent probability distributions over actions [Dulac-Arnold et al. 2019; Sutton and Barto 2018]. To accommodate for stochasticity, value-based methods resort to ϵ-greedy approaches, which are ad hoc and not a learned methodology [Kaelbling et al. 1996; Sutton and Barto 2018]. Finally, Policy-based methods are often used when we want to optimize a policy across an entire ML pipeline[Chen et al. 2022; Sutton and Barto 2018]. It is common for ad-ranking systems to start with a transformer-based encoder for ad retrieval and to conclude with a multi-head attention blending layer that predicts a downstream action, where end-to-end differentiability is required to allow the gradient to flow and therefore learn a cohesive policy [Pancha et al. 2022; Xia et al. 2023; Zhao et al. 2021]. This is difficult to achieve with value-based methods, which estimate a value function $Q(s, a)$ and then select actions via $\arg\max_a Q(s, a)$ [Dulac-Arnold et al. 2019; Sutton and Barto 2018]. Because arg max is non-differentiable, it breaks gradient flow in neural networks [Dulac-Arnold et al. 2019; Sutton and Barto 2018]. In contrast, policy-based methods directly parameterize the policy $\pi_\theta(a \mid s)$ using differentiable functions (e.g., softmax or Gaussians) [Sutton and Barto 2018; Williams 1992]. As a result, gradients can backpropagate through the entire pipeline, from the final action probabilities to earlier neural layers, enabling a fully integrated, end-to-end approach that can incorporate embeddings, attention mechanisms, and encoder-decoder architectures [Chen et al. 2022; Xia et al. 2023; Zhao et al. 2021].

While policy-based methods are powerful and overcome many of the limitations of value-based methods, policy-based methods also have their shortcomings [Levine et al. 2020; Sutton and Barto 2018]. Policy-based methods assume that the performance of the gradient can be accurately estimated and optimized through sampling [Sutton and Barto 2018; Williams 1992]. However, these estimates often suffer from high variance, which can lead to unstable learning and slow convergence [Schulman et al. 2015; Sutton and Barto 2018]. Because of the absence of an explicit value function, policy-based methods tend to require a larger number of samples compared to value-based methods, making them less sample-efficient [Dulac-Arnold et al. 2019; Levine et al. 2020]. To overcome these limitations, many large-scale production systems adopt hybrid approaches that combine both value-based and policy-based learning, most notably in the form of actor-critic architectures [Chen et al. 2022; Gauci et al. 2018; Sutton and Barto 2018]. In these methods, the actor is responsible for learning the policy, while the critic estimates the value function to guide and stabilize the policy updates [Sutton and Barto 2018; Williams 1992]. This combination improves learning stability, speeds up convergence, and enhances interpretability, making it well-suited for complex, real-world applications such as recommendation and ad-serving systems [Chen et al. 2022; Dulac-Arnold et al. 2019; Gauci et al. 2018].

To illustrate the distinction between value-based and policy-based methods in a simple example, consider the FrozenLake environment [Gym Documentation Team 2024]. In a value-based approach, the agent learns a state-action value function $Q(s, a)$ that estimates the expected cumulative reward for each action at every grid position [Kaelbling et al. 1996; Sutton and Barto 2018]. Once trained, the agent derives its policy by selecting the action with the highest value in each

state [Sutton and Barto 2018]. For example, from a particular icy tile near a hole, it may learn that moving right has a higher expected return than moving down, based on accumulated experience. In contrast, a policy-based agent would directly learn a parameterized policy $\pi(a \mid s)$ that outputs a probability distribution over actions for each state [Sutton and Barto 2018; Williams 1992]. Instead of estimating the value of each action, it adjusts its parameters to increase the likelihood of action sequences that successfully reach the goal [Sutton and Barto 2018; Williams 1992]. Over time, this agent may learn, for instance, that taking a probabilistic mix of "right" and "down" from a slippery tile leads to more consistent success, even if it cannot explicitly quantify the expected value of each option.

The value-based vs policy-based taxonomy offers a useful lens for categorizing RL approaches in ad policy optimization, based on how decisions are learned to maximize cumulative rewards [Lin et al. 2023; Sutton and Barto 2018; Zhao et al. 2021]. Platforms like LinkedIn and Google Ads have applied value-based approaches like Q-learning to optimize between discrete ad policies, such as selecting between low or high ad load treatments, by estimating the expected value of each action for a given user segment [McMahan et al. 2013; Theocharous et al. 2015; Zhao and Liu 2024]. Value-based learning works well when the action space is discrete, small, and easily enumerable, allowing the agent to derive a policy through the maximization of the learned Q-function [Dulac-Arnold et al. 2019; Sutton and Barto 2018]. In contrast, policy-based approaches are better suited for continuous, high-dimensional, or unstructured decision spaces [Dulac-Arnold et al. 2019; Schulman et al. 2015; Sutton and Barto 2018]. While industry examples above from Google and LinkedIn have applied value-based methods such as Q-learning to optimize among a small set of discrete ad policies (e.g., choosing between high and low ad load treatments) [McMahan et al. 2013; Theocharous et al. 2015], it is also possible to optimize for the ad policy directly in a more flexible formulation [Sutton and Barto 2018; Williams 1992; Zhao et al. 2021]. For instance, rather than predefining a limited set of treatment arms, TikTok used Proximal Policy Optimization (PPO) to learn a stochastic ranking policy that directly maps user context to slates of ads [Zhao et al. 2021; Zhao and Liu 2024]. This approach can handle potentially large or complex sets of ads more flexibly, allowing for direct parameterization of the stochastic policy [Dulac-Arnold et al. 2019; Ie et al. 2019b; Zhao et al. 2021]. The built-in stochasticity facilitates exploration (helping to uncover effective ad configurations) and adaptation to shifting user behaviors, without constraining the policy to a small, predefined action space [Sutton and Barto 2018; Williams 1992; Zhao et al. 2021].

Actor-critic architectures present a hybrid method that leverages the benefits of both value-based and policy-based methods [Sutton and Barto 2018]. The actor learns the policy, while the critic provides value estimates to stabilize and accelerate learning [Lin et al. 2023; Sutton and Barto 2018]. The critic acts as a baseline to reduce the high variance of policy gradient methods (e.g. REINFORCE), and the actor's explicit policy handles large or complex action spaces better than value-based approaches (e.g. DQN) [Chen et al. 2022; Mnih et al. 2016; Sutton and Barto 2018].

There are well-documented industry examples of the use of actor-critic methods in ad recommendations. For example, Facebook reported that RL models (including off-policy actor-critic methods) trained on its Horizon platform outperformed and even replaced previous supervised recommendation systems in tasks like notifications and video ranking [Gauci et al. 2018]. YouTube introduced an off-policy actor-critic approach to augment their previous REINFORCE agent (policy-gradient based) by adding a critic network that estimates Q-values via temporal difference (TD) learning [Chen et al. 2022]. The use of Q-value estimates enables lower-variance policy updates compared to Monte Carlo returns; one-step importance correction offers more robust handling of off-policy data than full-trajectory importance sampling; and bootstrapping with one-step TD targets improves the estimation of long-term rewards [Chen et al. 2022].

Actor-critic methods are particularly useful in multi-objective settings where both revenue and user engagement must be balanced, and where using logged data for offline training is desired [Cai et al. 2023]. To this end, two-staged constrained actor-critic methods have been effectively used to optimize the main objective while enforcing guardrail constraints through a constrained MDP formulation [Cai et al. 2023]. In stage one, multiple actor-critic agents learn to optimize auxiliary objectives (e.g. engagement such as likes, saves, and shares) [Cai et al. 2023]. In stage two, a primary actor-critic learns a policy maximizing the main objective (e.g. revenue, ad clicks) while staying close to the auxiliary policies to satisfy those constraints. This two-tier actor-critic approach achieved a superior balance: offline experiments showed it outperformed alternative RL methods on the trade-off between watch time and other interaction metrics [Cai et al. 2023].

Actor-critic RL brings the bias–variance trade-off under control: the critic (value-based element) introduces bias but provides low-variance evaluation, and the actor (policy-based element) ensures we still directly optimize the true objective [Chen et al. 2022; Sutton and Barto 2018]. Understanding the structure and cardinality of the action and state spaces, the importance of calibrating exploration vs exploitation, the available computational budget, and the degree of cohesion required across an ML pipeline are some key factors that help determine which paradigm to use in practice [Dulac-Arnold et al. 2019; Schulman et al. 2017; Zhu et al. 2021].

4.6 Balancing Exploration and Exploitation in Policy Learning

Balancing exploration and exploitation is a fundamental challenge in policy learning [Sutton and Barto 2018], particularly at large scale, whereby exhaustively exploring all options is computationally infeasible, the cost of suboptimal decisions compounds quickly, and sustained bias toward early winners can prevent the discovery of truly optimal policies over time [Dulac-Arnold et al. 2021].

In ad policy optimization, tuning exploration and exploitation is critical across multiple dimensions, notably ad selection and ad load. In *ad selection*, a system must decide between exploring new ads that might yield higher rewards and exploiting ads known to perform well [Auer et al. 2002; Bietti et al. 2021; Russo et al. 2018]. In setting *ad load* configuration, platforms must weigh the exploration of higher ad exposure for queries with high commercial intent against the risk of degrading user experience [Carrion et al. 2021; Hohnhold et al. 2015b; Zhang et al. 2018]. While exploring more aggressive ad load configurations may unlock revenue gains, it risks triggering ad fatigue, reduced engagement, and eventual user attrition [Sagtani et al. 2023; Silberstein et al. 2023].

In this section, we introduce the commonly used methods in industry as well as their pros and cons, and the contexts in which they are most effective.

4.6.1 ***ϵ-Greedy Strategy.*** One of the simplest and most widely used exploration methods, the epsilon-greedy strategy allows the agent to primarily exploit known successful actions (with probability $1-\epsilon$) while occasionally exploring random actions (with probability ϵ) [Russell and Norvig 2016; Sutton and Barto 2018]. Over time, ϵ *can* be reduced, shifting focus toward exploitation as the agent gains more experience [Russo et al. 2018; Sutton and Barto 2018]. However, deciding how much and when to decrease ϵ is non-trivial, as reducing exploration too quickly can lead to premature convergence on suboptimal actions [Dulac-Arnold et al. 2019; Sutton and Barto 2018].

ϵ-Greedy based exploration/exploitation is a relatively easy method to implement, computationally efficient, and thus well-suited for real-time applications [Russell and Norvig 2016; Sutton and Barto 2018]. However, because it treats all actions uniformly during exploration, ϵ-greedy does not differentiate between tried and untried actions, account for uncertainty in value estimates, or prioritize actions with higher potential expected improvement [Dulac-Arnold et al. 2019; Sutton

and Barto 2018]. This can lead to missed opportunities, especially in environments with large action spaces, because as the number of possible actions increases, the probability of selecting any particular action during exploration decreases [Bubeck et al. 2012; Sutton and Barto 2018]. As a result, many potentially high-value actions may never be explored, or may be explored too infrequently to accurately estimate their value [Dulac-Arnold et al. 2019; Sutton and Barto 2018]. ϵ-greedy is most effective in smaller action spaces, where exploration remains tractable and the risk of encountering extremely low-value actions is lower due to the reduced likelihood of extreme outcomes [Dulac-Arnold et al. 2019; Sutton and Barto 2018].

4.6.2 ***Upper Confidence Bound (UCB).*** In contrast to ϵ-greedy strategies, which explore uniformly at random and do not incorporate knowledge of an action's estimated value or the uncertainty in that estimate, the UCB algorithm ranks actions using a composite score that integrates both expected reward and uncertainty [Auer et al. 2002]. At each time step t, UCB selects the action a that maximizes:

$$\mathrm{UCB}_t(a) = \hat{Q}_t(a) + \alpha \cdot \sqrt{\frac{\log t}{N_t(a)}} \tag{4}$$

where the first term $\hat{Q}_t(a)$ is the estimated average reward of action a, and the second term is called the exploration bonus, which reflects the uncertainty in the reward estimate [Auer et al. 2002; Bubeck et al. 2012]. $N_t(a)$ is the number of times action a has been selected, which shrinks the bonus as the action is chosen more often and reduces exploration as a result [Auer et al. 2002; Bubeck et al. 2012]. Similarly, $\log t$ grows slowly over time, which ensures continued exploration in later rounds. This formulation with the exploration and exploitation terms ensures that actions are prioritized either because they have performed well (high $\hat{Q}_t(a)$) or because they are underexplored (high uncertainty bonus) [Auer et al. 2002; Bubeck et al. 2012]. α is a tunable parameter that determines how much exploration is valued over exploitation [Auer et al. 2002; Bubeck et al. 2012; Sutton and Barto 2018]. As an action is selected more frequently, its estimate becomes more reliable, uncertainty decreases, and the confidence interval narrows [Auer et al. 2002; Bubeck et al. 2012]. As a result, the UCB score may increase or decrease: if the action is good, its estimated reward improves and the score may remain high or even rise despite the shrinking bonus; if the action is bad, the estimate stays low and the diminishing uncertainty term no longer inflates its score, causing it to be deprioritized over time [Auer et al. 2002; Bubeck et al. 2012].

The unified scoring function in UCB naturally balances exploration and exploitation [Auer et al. 2002; Bubeck et al. 2012]. It allows the agent to identify promising but underexplored actions while avoiding those that are confidently known to perform poorly [Auer et al. 2002]. While UCB provides tighter bounds on long-term regret, it can be more computationally expensive than simpler methods like ϵ-greedy and may over-explore suboptimal actions with high uncertainty, leading to short-term performance losses [Bubeck et al. 2012; Dulac-Arnold et al. 2019; Sutton and Barto 2018].

LinkedIn used UCB-style methods to optimize ad placement and pacing in user feeds. In the early stages of their system, UCB was employed to evaluate which treatment arms, such as different spacing, reserve price, and ad load configurations, performed best across user segments. UCB helped balance exploration of new pacing parameters with exploitation of those already showing promising results [Yan et al. 2020]. Microsoft, through the Contextual Bandit Bake-Off, found that UCB and its linear variant, LinUCB, were competitive or superior to ϵ-greedy approaches across multiple offline datasets for ad personalization and content recommendation [Bietti et al. 2021].

Facebook (Meta) incorporated UCB-inspired bandit methods within its Horizon RL platform, particularly for notification ranking and ad sequencing tasks. These methods were used in early-stage exploration, particularly in cold-start scenarios or on surfaces with limited historical data, before transitioning to more complex actor-critic architectures for long-term optimization [Gauci et al. 2018].

UCB performs best in environments where data is sparse, optimization horizons are long, and the cost of regret is high, making it well-suited for ad policy optimization where long-term performance (like maximizing user lifetime value or engagement) is key [Bubeck et al. 2012; Dulac-Arnold et al. 2019; Theocharous et al. 2015]. However, in real-time applications where immediate results are prioritized, UCB's tendency to over-explore may be less suitable [Dulac-Arnold et al. 2019; McMahan et al. 2013; Sutton and Barto 2018]. Additionally, the choice of the exploration parameter α can be brittle as too conservative a setting may limit the discovery of better actions, while too aggressive a setting may amplify noise and degrade performance.

4.6.3 ***Thompson Sampling***. Thompson Sampling (TS) uses a Bayesian approach in which actions are selected by sampling from the posterior distribution over rewards and choosing the action with the highest sampled value [Russo et al. 2018; Sutton and Barto 2018]. Formally, the selected action at time t is:

$$a_t = \arg\max_{a \in \mathcal{A}} \tilde{Q}_t(a) \tag{5}$$

Formally, the selected action at time t is: where $\mathcal{A}$ is the action set and $\tilde{Q}_t(a)$ is a sample from the posterior distribution of the expected reward for action a at time t [Russo et al. 2018].

While each individual decision is based on a single random sample, across many realizations, the likelihood that an action is selected reflects the posterior probability that it is the optimal choice [Russo et al. 2018]. Initially, TS relies on a prior distribution that reflects the initial belief about the reward of each action, usually based on some default assumptions or domain knowledge [Russo et al. 2018]. As more data is collected through interactions (e.g., clicks or conversions in an ad setting), the prior is updated to form a posterior distribution, representing the system's refined belief about expected rewards [Russo et al. 2018]. Actions with greater uncertainty (i.e., wider posterior distributions) are more likely to generate extreme reward samples, occasionally leading to their selection even if their mean reward is lower [Russo et al. 2018]. In contrast, actions with higher expected rewards and narrower posterior distributions are selected more consistently due to their reliably higher sampled values [Russo et al. 2018]. Thus, TS leverages posterior variance to simultaneously enable exploration and exploitation without requiring an explicit exploration bonus as used in UCB methods [Russo et al. 2018].

While both UCB and TS leverage estimates of variability to manage exploration and exploitation, they differ in how they quantify uncertainty, handle randomness, and select actions [Russo et al. 2018; Sutton and Barto 2018]. UCB constructs an upper confidence bound for each action and always selects the action with the highest upper bound [Auer et al. 2002], whereas TS samples from the posterior distribution over rewards and chooses the action with the highest sampled value [Russo et al. 2018]. As a result, UCB deterministically selects the same action given the same inputs, while TS's choice varies across runs due to its sampling-based nature [Russo et al. 2018]. Although UCB is often easier to implement, as it does not require specifying prior or posterior distributions, it can be more brittle to tune (e.g., selecting an appropriate exploration constant α) [Auer et al. 2002; Sutton and Barto 2018]. In contrast, TS is generally more robust to hyperparameter choices and tends to perform better in non-stationary environments [Chapelle and Li 2011; Russo et al. 2018].

Studies by Chapelle and Li [Chapelle and Li 2011] show that TS can perform as well as, or better than, UCB in terms of cumulative regret, defined as the total loss incurred from not consistently selecting the optimal action [Chapelle and Li 2011; Russo et al. 2018]. This is especially true in scenarios where data is sparse or feedback is delayed [Chapelle and Li 2011]. In contrast to UCB, which might over-explore uncertain actions with low potential returns, TS ensures a more balanced and probabilistic form of exploration [Russo et al. 2018]. TS is naturally well-suited for industry problems like selecting ads for new or existing users, particularly in dynamic environments such as online advertising, where user preferences shift frequently and new content is introduced continuously. However, because TS relies on randomized sampling, it can exhibit high variance, which may lead to less stable performance in environments that prioritize short-term accuracy or require highly reliable decision-making [Russo et al. 2018].

4.6.4 ***Broader Considerations for Exploration and Exploitation.*** While the previous discussion focused on canonical exploration strategies that are widely deployed and highly effective in industrial recommender and ad systems (e.g., ϵ-Greedy, UCB, Thompson Sampling), it is also important to recognize that additional techniques are often employed to address more complex, high-dimensional, or rapidly evolving environments. For instance, in dynamic ad allocation systems, parameters are often tuned in real-time to balance short-term revenue generation against user experience metrics such as Click Yield, which measures the rate of user interactions per ad impression [Zhang et al. 2018]. By adjusting trade-off parameters in response to shifting engagement patterns, platforms can dynamically prioritize either immediate monetization or sustained user satisfaction. Moreover, alternative approaches such as entropy-regularized policies [Mnih et al. 2016], Bayesian optimization [Shahriari et al. 2015], and policy gradient methods like Proximal Policy Optimization (PPO) [Schulman et al. 2017] extend the exploration-exploitation framework in settings where continuous adaptation is critical.

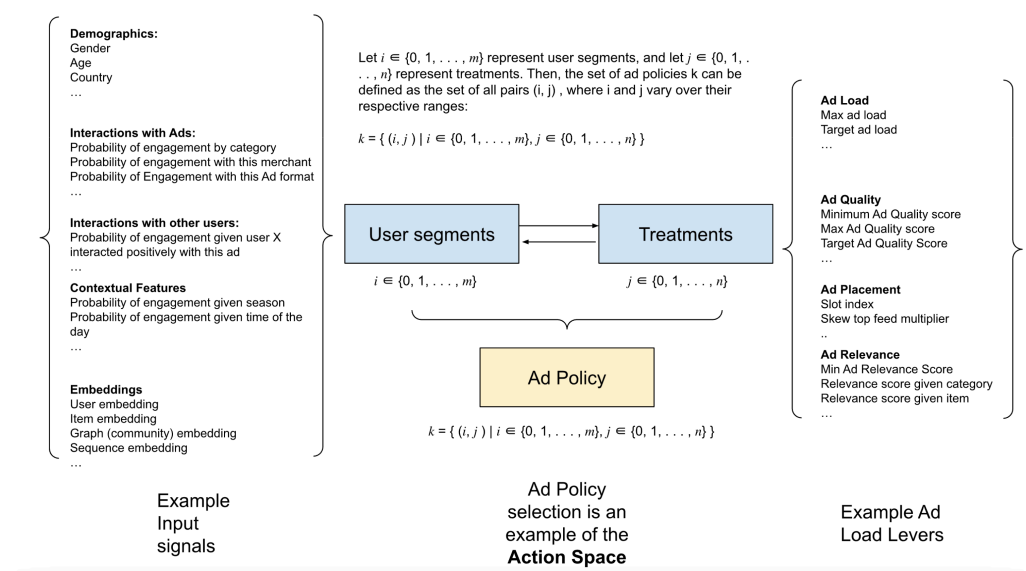

Fig. 2. Example of an action set in the context of ad policies in recommendation systems

Method	How It Works	Pros	Cons	Best Use Cases
Epsilon-Greedy	Selects the best-known action with probability $1 - \epsilon$ and explores randomly with probability ϵ.	• Simple to implement • Low computational cost • Easy to understand	• Fixed exploration rate • Might miss better actions • Not adaptive to context	• Simple, static environments • Prioritize computational efficiency
Upper Confidence Bound (UCB)	Selects actions based on confidence intervals combining reward estimates and uncertainty.	• Systematic exploration • Minimizes long-term regret • Strong theoretical foundations	• May overexplore uncertain actions • Higher computational complexity	• Real-time ad placement • Long-term optimization
Regret Confidence Bounds (RegCB)	Extends UCB by using regression oracles to compute confidence intervals based on contextual features.	• Handles high-dimensional, sparse data • More accurate reward estimates • Well-suited for complex environments	• High computational cost • Difficult to scale	• High-dimensional environments • Needs precise reward prediction
Thompson Sampling (TS)	Uses Bayesian sampling from posterior distributions to estimate optimal actions.	• Adapts quickly to new data • Naturally balances exploration • Effective for cold-start problems	• Can exhibit high variance • Moderate computational complexity	• Cold-start and sparse data • Dynamic, shifting environments

Table 8. Comparison of Exploration-Exploitation Methods in Ad Policy Optimization

5 CONCLUSIONS

The integration of RL in policy optimization for large-scale recommender systems has gained popularity as a promising approach to balance long-term and short-term objectives in environments with complex user behavior and revenue dynamics. However, scaling RL in practice presents challenges, including vast state and action spaces, reliance on proxy metrics, and significant computational demands. This paper provides an overview of effective, scalable techniques that address these challenges, offering insights into how RL can be applied at scale. Unlike traditional methods that focus on optimizing short-term outcomes, RL's cumulative reward framework enables it to account for both immediate and delayed feedback. Together, contextual bandits and offline replay offer scalable, interpretable solutions that approximate full RL while managing real-time constraints. Contextual bandits support selecting optimal actions within each interaction based on the current state, maximizing immediate rewards, which are typically proxy metrics of the true long-term reward. Offline replay complements this by enabling counterfactual learning: multiple treatments can be tested across randomized populations to evaluate causal effects, refining policies for diverse user segments.

Future research may focus on enhancing interpretability, refining reward functions, and exploring decentralized approaches to handle complex decision layers. These advancements will bring RL closer to fulfilling its potential in providing tailored, dynamic ad recommendations that align user satisfaction with business goals in large-scale recommendation systems.

ACKNOWLEDGMENTS

I would like to thank Adam Obeng, Bee-Chung Chen, and Jay Adams for their valuable guidance, critical coaching, and insights, which have significantly shaped my approach and understanding of industry practices. Their expertise has been instrumental in aligning academic research with practical applications.

REFERENCES

Zoë Abrams and Erik Vee. 2007. Personalized ad delivery when ads fatigue: An approximation algorithm. In *International Workshop on Web and Internet Economics*. Springer, 535–540.

M Mehdi Afsar, Trafford Crump, and Behrouz Far. 2022. Reinforcement learning based recommender systems: A survey. *Comput. Surveys* 55, 7 (2022), 1–38.

Alekh Agarwal, Daniel Hsu, Satyen Kale, John Langford, Lihong Li, and Robert Schapire. 2014. Taming the monster: A fast and simple algorithm for contextual bandits. In *International conference on machine learning*. PMLR, 1638–1646.

Deepak Agarwal, David Cornell, Bin Huang, Bee-Chung Chen Lee, Ruoxi Tang, Le Zhang, Avery Wolf, Yuchen Shi, Ye Wang, Yin Guo, Srijan Kumar Garimella, Nikita Korolko, and Sandeep Muthukrishnan. 2019. Online Parameter Selection for Web-based Services. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD '19)*. 1645–1653.

Deepak K Agarwal and Bee-Chung Chen. 2016. *Statistical methods for recommender systems*. Cambridge University Press.

Rishabh Agarwal, Dale Schuurmans, and Mohammad Norouzi. 2020. An Optimistic Perspective on Offline Reinforcement Learning. In *International Conference on Machine Learning (ICML)*. 104–114.

Zijian An and Inwhee Joe. 2024. TMH: Two-Tower Multi-Head Attention neural network for CTR prediction. *Plos one* 19, 3 (2024), e0295440.

Peter Auer, Nicolo Cesa-Bianchi, and Paul Fischer. 2002. Finite-Time Analysis of the Multiarmed Bandit Problem. *Machine learning* 47 (2002), 235–256.

Joel Barajas, Narayan Bhamidipati, and James G Shanahan. 2022. Online advertising incrementality testing: Practical lessons, paid search and Emerging challenges. In *European Conference on Information Retrieval*. Springer, 575–581.

Oren Barkan and Noam Koenigstein. 2016. Item2vec: neural item embedding for collaborative filtering. In *2016 IEEE 26th international workshop on machine learning for signal processing (MLSP)*. IEEE, 1–6.

Yoshua Bengio, Patrice Simard, and Paolo Frasconi. 1994. Learning Long-Term Dependencies with Gradient Descent is Difficult. *IEEE Transactions on Neural Networks* 5, 2 (1994), 157–166. https://www.cs.toronto.edu/~hinton/absps/bengio+al94.pdf

James Bennett and Stan Lanning. 2007. The netflix prize. (2007).

Alberto Bietti, Alekh Agarwal, and John Langford. 2021. A contextual bandit bake-off. *Journal of Machine Learning Research* 22, 133 (2021), 1–49.

Jesus Bobadilla, Fernando Ortega, Antonio Hernando, and Arturo Guti 'errez. 2013. Recommender systems survey. *Knowledge-Based Systems* 46 (2013), 109–132. https://doi.org/10.1016/j.knosys.2013.03.012

Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. 2016. OpenAI Gym. *arXiv preprint arXiv:1606.01540* (2016). https://arxiv.org/abs/1606.01540 Accessed: 2025-03-28.

Sébastien Bubeck, Nicolo Cesa-Bianchi, et al. 2012. Regret analysis of stochastic and nonstochastic multi-armed bandit problems. *Foundations and Trends® in Machine Learning* 5, 1 (2012), 1–122.

Giuseppe Burtini, Jason L. Loeppky, and Ramon Lawrence. 2015. Improving Online Marketing Experiments with Drifting Multi-armed Bandits. In *Proceedings of the 7th International Conference on Agents and Artificial Intelligence (ICAART)*, Vol. 1. SCITEPRESS, 135–146.

Qingpeng Cai, Zhenghai Xue, Chi Zhang, Wanqi Xue, Shuchang Liu, Ruohan Zhan, Xueliang Wang, Tianyou Zuo, Wentao Xie, Dong Zheng, et al. 2023. Two-stage constrained actor-critic for short video recommendation. In *Proceedings of the ACM Web Conference 2023*. 865–875.

Carlos Carrion, Zenan Wang, Harikesh Nair, Xianghong Luo, Yulin Lei, Xiliang Lin, Wenlong Chen, Qiyu Hu, Changping Peng, Yongjun Bao, et al. 2021. Blending advertising with organic content in e-commerce: A virtual bids optimization approach. *arXiv preprint arXiv:2105.13556* (2021).

Fatih Çelik, Mehmet Safa Çam, and Mehmet Ali Koseoglu. 2023. Ad avoidance in the digital context: A systematic literature review and research agenda. *International Journal of Consumer Studies* 47, 6 (2023), 2071–2105.

Deepayan Chakrabarti, Deepak Agarwal, and Vanja Josifovski. 2008. Contextual advertising by combining relevance with click feedback. In *Proceedings of the 17th international conference on World Wide Web*. 417–426.

Olivier Chapelle and Lihong Li. 2011. An empirical evaluation of thompson sampling. *Advances in neural information processing systems* 24 (2011).

Bee-Chung Chen, Dmitry Pavlov, John F Canny, and Lodewijk Rijckaert. 2009. Large-scale Behavioral Targeting. In *Proceedings of the 15th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD '09)*. 209–218.

Minmin Chen, Can Xu, Vince Gatto, Devanshu Jain, Aviral Kumar, and Ed Chi. 2022. Off-policy actor-critic for recommender systems. In *Proceedings of the 16th ACM Conference on Recommender Systems*. 338–349.

Heng-Tze Cheng, Levent Koc, Jeremiah Harmsen, Tal Shaked, Tushar Chandra, Hrishi Aradhye, Glen Anderson, Greg Corrado, Wei Chai, Mustafa Ispir, et al. 2016. Wide & deep learning for recommender systems. In *Proceedings of the 1st workshop on deep learning for recommender systems*. 7–10.

Paul Covington, Jay Adams, and Emre Sargin. 2016. Deep neural networks for youtube recommendations. In *Proceedings of the 10th ACM conference on recommender systems*. 191–198.

Qing Cui, Feng-Shan Bai, Bin Gao, and Tie-Yan Liu. 2015. Global optimization for advertisement selection in sponsored search. *Journal of Computer Science and Technology* 30 (2015), 295–310.

Cristian Danescu-Niculescu-Mizil, Andrei Z Broder, Evgeniy Gabrilovich, Vanja Josifovski, and Bo Pang. 2010. Competing for users' attention: on the interplay between organic and sponsored search results. In *Proceedings of the 19th international conference on World wide web*. 291–300.

Christos Dimitrakakis and Ronald Ortner. 2018. Decision making under uncertainty and reinforcement learning. *Book available at http://www. cse. chalmers. se* (2018).

Miroslav Dudík, John Langford, and Lihong Li. 2011. Doubly robust policy evaluation and learning. *arXiv preprint arXiv:1103.4601* (2011).

Gabriel Dulac-Arnold, Nir Levine, Daniel J Mankowitz, Jerry Li, Cosmin Paduraru, Sven Gowal, and Todd Hester. 2021. Challenges of real-world reinforcement learning: definitions, benchmarks and analysis. *Machine Learning* 110, 9 (2021), 2419–2468.

Gabriel Dulac-Arnold, Daniel Mankowitz, and Todd Hester. 2019. Challenges of real-world reinforcement learning. *arXiv preprint arXiv:1904.12901* (2019).

Andrew Ellam and Marco Ottaviani. 2003. Overture and Google: Internet Pay-Per-Click (PPC) Advertising Auctions. *London Business School Case Study CS-03-022* (2003).

Yagil Engel and David Maxwell Chickering. 2008. Incorporating user utility into sponsored-search auctions. In *Proceedings of the 7th international joint conference on Autonomous agents and multiagent systems-Volume 3*. 1565–1568.

Tom Everitt, Marcus Hutter, Ramana Kumar, and Victoria Krakovna. 2021. Reward tampering problems and solutions in reinforcement learning: A causal influence diagram perspective. *Synthese* 198, Suppl 27 (2021), 6435–6467.

Daniel C Fain and Jan O Pedersen. 2006. Sponsored search: A brief history. *Bulletin-American Society For Information Science And Technology* 32, 2 (2006), 12.

João Gama, Indrė Žliobaitė, Albert Bifet, Mykola Pechenizkiy, and Abdelhamid Bouchachia. 2014. A survey on concept drift adaptation. *ACM computing surveys (CSUR)* 46, 4 (2014), 1–37.

Jason Gauci, Edoardo Conti, Yitao Liang, Kittipat Virochsiri, Yuchen He, Zachary Kaden, Vivek Narayanan, Xiaohui Ye, Zhengxing Chen, and Scott Fujimoto. 2018. Horizon: Facebook's open source applied reinforcement learning platform. *arXiv preprint arXiv:1811.00260* (2018).

Claire Glanois, Paul Weng, Matthieu Zimmer, Dong Li, Tianpei Yang, Jianye Hao, and Wulong Liu. 2024. A survey on interpretable reinforcement learning. *Machine Learning* (2024), 1–44.

Carlos A Gomez-Uribe and Neil Hunt. 2015. The netflix recommender system: Algorithms, business value, and innovation. *ACM Transactions on Management Information Systems (TMIS)* 6, 4 (2015), 1–19.

Mihajlo Grbovic and Haibin Cheng. 2018. Real-time personalization using embeddings for search ranking at airbnb. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*. 311–320.

Gym Documentation Team. 2024. Frozen Lake Environment in OpenAI Gym. https://www.gymlibrary.dev/environments/toy_text/frozen_lake/. Accessed: 23 August 2024.

Xiangnan He, Lizi Liao, Hanwang Zhang, Liqiang Nie, Xia Hu, and Tat-Seng Chua. 2017. Neural collaborative filtering. In *Proceedings of the 26th international conference on world wide web*. 173–182.

Xinran He, Junfeng Pan, Ou Jin, Tianbing Xu, Bo Liu, Tao Xu, Yanxin Shi, Antoine Atallah, Ralf Herbrich, Stuart Bowers, et al. 2014. Practical lessons from predicting clicks on ads at facebook. In *Proceedings of the eighth international workshop on data mining for online advertising*. 1–9.

Henning Hohnhold, Deirdre O'Brien, and Diane Tang. 2015a. Focus on the Long-Term: It's better for Users and Business. In *Proceedings 21st Conference on Knowledge Discovery and Data Mining*. Sydney, Australia. http://dl.acm.org/citation.cfm?doid=2783258.2788583

Henning Hohnhold, Deirdre O'Brien, and Diane Tang. 2015b. Focusing on the long-term: It's good for users and business. In *Proceedings of the 21th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 1849–1858.

Yu Jeffrey Hu. 2004. Performance-based pricing models in online advertising. *Available at SSRN 501082* (2004).

Eugene Ie, Chih-wei Hsu, Martin Mladenov, Vihan Jain, Sanmit Narvekar, Jing Wang, Rui Wu, and Craig Boutilier. 2019a. Recsim: A configurable simulation platform for recommender systems. *arXiv preprint arXiv:1909.04847* (2019).

Eugene Ie, Vihan Jain, Jing Wang, Sanmit Narvekar, Ritesh Agarwal, Rui Wu, Heng-Tze Cheng, Tushar Chandra, and Craig Boutilier. 2019b. SlateQ: A Tractable Decomposition for Reinforcement Learning with Recommendation Sets.. In *IJCAI*, Vol. 19. 2592–2599.

Eugene Ie, Vihan Jain, Jing Wang, Sanmit Narvekar, Ritesh Agarwal, Rui Wu, Heng-Tze Cheng, Morgane Lustman, Vince Gatto, Paul Covington, et al. 2019c. Reinforcement learning for slate-based recommender systems: A tractable decomposition and practical methodology. *arXiv preprint arXiv:1905.12767* (2019).

Dietmar Jannach and Himan Abdollahpouri. 2023. A Survey on Multi-Objective Recommender Systems. *Frontiers in Big Data* 6 (2023). https://doi.org/10.3389/fdata.2023.1157899

Leslie Pack Kaelbling, Michael L Littman, and Anthony R Cassandra. 1998. Planning and acting in partially observable stochastic domains. *Artificial intelligence* 101, 1-2 (1998), 99–134.

Leslie Pack Kaelbling, Michael L Littman, and Andrew W Moore. 1996. Reinforcement learning: A survey. *Journal of artificial intelligence research* 4 (1996), 237–285.

Wang-Cheng Kang and Julian McAuley. 2018a. Self-Attentive Sequential Recommendation. In *2018 IEEE International Conference on Data Mining (ICDM)*. 197–206. https://doi.org/10.1109/ICDM.2018.00035

Wang-Cheng Kang and Julian McAuley. 2018b. Self-Attentive Sequential Recommendation. In *Proceedings of the 2018 IEEE International Conference on Data Mining (ICDM)*. 197–206. https://doi.org/10.1109/ICDM.2018.00035

Tanya Kant. 2021. A History of the Data-Tracked User. *The MIT Press Reader* (2021). https://thereader.mitpress.mit.edu/a-history-of-the-data-tracked-user/

Yehuda Koren. 2008. Factorization meets the neighborhood: a multifaceted collaborative filtering model. In *Proceedings of the 14th ACM SIGKDD international conference on Knowledge discovery and data mining*. 426–434.

Yehuda Koren, Robert Bell, and Chris Volinsky. 2009. Matrix factorization techniques for recommender systems. *Computer* 42, 8 (2009), 30–37. https://doi.org/10.1109/MC.2009.263

Iordanis Koutsopoulos and Panagiotis Spentzouris. 2016. Native advertisement selection and allocation in social media post feeds. In *Machine Learning and Knowledge Discovery in Databases: European Conference, ECML PKDD 2016, Riva del Garda, Italy, September 19-23, 2016, Proceedings, Part I 16*. Springer, 588–603.

Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. 2012. Imagenet classification with deep convolutional neural networks. *Advances in neural information processing systems* 25 (2012).

Volodymyr Kuleshov and Doina Precup. 2014. Algorithms for Multi-Armed Bandit Problems. *arXiv preprint arXiv:1402.6028* (2014). https://arxiv.org/abs/1402.6028

Nathan Lambert, Thomas Krendl Gilbert, and Tom Zick. 2023. Entangled preferences: The history and risks of reinforcement learning and human feedback. *arXiv preprint arXiv:2310.13595* (2023).

Sergey Levine, Aviral Kumar, George Tucker, and Justin Fu. 2020. Offline reinforcement learning: Tutorial, review, and perspectives on open problems. *arXiv preprint arXiv:2005.01643* (2020).

Lihong Li, Wei Chu, John Langford, and Robert E Schapire. 2010. A contextual-bandit approach to personalized news article recommendation. In *Proceedings of the 19th international conference on World wide web*. 661–670.

Yuanguo Lin, Yong Liu, Fan Lin, Lixin Zou, Pengcheng Wu, Wenhua Zeng, Huanhuan Chen, and Chunyan Miao. 2023. A survey on reinforcement learning for recommender systems. *IEEE Transactions on Neural Networks and Learning Systems* (2023).

Greg Linden, Brent Smith, and Jeremy York. 2003. Amazon. com recommendations: Item-to-item collaborative filtering. *IEEE Internet computing* 7, 1 (2003), 76–80.

Wenzhang Liu, Lianjun Jin, Lu Ren, Chaoxu Mu, and Changyin Sun. 2025. Reducing Action Space for Deep Reinforcement Learning via Causal Effect Estimation. *arXiv preprint arXiv:2501.14543* (2025).

Zhuoran Liu, Leqi Zou, Xuan Zou, Caihua Wang, Biao Zhang, Da Tang, Bolin Zhu, Yijie Zhu, Peng Wu, Ke Wang, et al. 2022. Monolith: real time recommendation system with collisionless embedding table. *arXiv preprint arXiv:2209.07663* (2022).

Shanshan Lyu, Qiwei Chen, Tao Zhuang, and Junfeng Ge. 2023. Entire Space Learning Framework: Unbias Conversion Rate Prediction in Full Stages of Recommender System. *arXiv preprint arXiv:2303.00276* (2023).

Jiaqi Ma, Zhe Zhao, Xinyang Yi, Jilin Chen, Lichan Hong, and Ed H Chi. 2018. Modeling task relationships in multi-task learning with multi-gate mixture-of-experts. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*. 1930–1939.

Bogdan Mazoure, Paul Mineiro, Pavithra Srinath, Reza Sharifi Sedeh, Doina Precup, and Adith Swaminathan. 2021. Improving long-term metrics in recommendation systems using short-horizon reinforcement learning. *arXiv preprint arXiv:2106.00589* (2021).

Thomas M McDonald, Lucas Maystre, Mounia Lalmas, Daniel Russo, and Kamil Ciosek. 2023a. Impatient bandits: Optimizing recommendations for the long-term without delay. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge*

Discovery and Data Mining. 1687–1697.

Thomas M. McDonald, Lucas Maystre, Mounia Lalmas, Daniel Russo, and Kamil Ciosek. 2023b. Optimizing for the Long-Term Without Delay. Spotify Research Blog. https://research.atspotify.com/2023/07/optimizing-for-the-long-term-without-delay/

H Brendan McMahan, Gary Holt, David Sculley, Michael Young, Dietmar Ebner, Julian Grady, Lan Nie, Todd Phillips, Eugene Davydov, Daniel Golovin, et al. 2013. Ad click prediction: a view from the trenches. In *Proceedings of the 19th ACM SIGKDD international conference on Knowledge discovery and data mining*. 1222–1230.

Rishabh Mehrotra, James McInerney, Hugues Bouchard, Mounia Lalmas, and Fernando Diaz. 2018. Towards a Fair Marketplace: Counterfactual Evaluation of the Trade-off between Relevance, Fairness & Satisfaction in Recommendation Systems. In *Proceedings of the 27th ACM International Conference on Information and Knowledge Management (CIKM '18)*.

Rishabh Mehrotra, Niannan Xue, and Mounia Lalmas. 2020. Bandit Based Optimization of Multiple Objectives on a Music Streaming Platform. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD '20)*. 3224–3233. https://doi.org/10.1145/3394486.3403392

Paul Michel, Omer Levy, and Graham Neubig. 2019. Are Sixteen Heads Really Better Than One?. In *Advances in Neural Information Processing Systems 32 (NeurIPS)*. 14014–14024.

Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S Corrado, and Jeff Dean. 2013. Distributed representations of words and phrases and their compositionality. *Advances in neural information processing systems* 26 (2013).

Volodymyr Mnih, Adria Puigdomenech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. 2016. Asynchronous methods for deep reinforcement learning. In *International conference on machine learning*. PMLR, 1928–1937.

Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, Stig Petersen, et al. 2015. Human-level control through deep reinforcement learning. *Nature* 518, 7540 (2015), 529–533. https://doi.org/10.1038/nature14236

Maxim Naumov, Dheevatsa Mudigere, Hao-Jun Michael Shi, Jianyu Huang, Narayanan Sundaraman, Jongsoo Park, Xiaodong Wang, Udit Gupta, Carole-Jean Wu, Alisson G Azzolini, et al. 2019. Deep learning recommendation model for personalization and recommendation systems. *arXiv preprint arXiv:1906.00091* (2019).

Nielsen. 2017. When It Comes to Advertising Effectiveness, What Is Key? Nielsen Insights Online Article. https://www.nielsen.com/insights/2017/when-it-comes-to-advertising-effectiveness-what-is-key/ Accessed: 2025-03-27.

Yunzhu Pan, Nian Li, Chen Gao, Jianxin Chang, Yanan Niu, Yang Song, Depeng Jin, and Yong Li. 2023. Learning and optimization of implicit negative feedback for industrial short-video recommender system. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*. 4787–4793.

Nikil Pancha, Andrew Zhai, Jure Leskovec, and Charles Rosenberg. 2022. Pinnerformer: Sequence modeling for user representation at pinterest. In *Proceedings of the 28th ACM SIGKDD conference on knowledge discovery and data mining*. 3702–3712.

Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. 2013. On the difficulty of training recurrent neural networks. In *Proceedings of the 30th International Conference on Machine Learning (ICML)*, Vol. 28. PMLR, 1310–1318. http://proceedings.mlr.press/v28/pascanu13.pdf Available at Proceedings of Machine Learning Research (PMLR).

Erika Puiutta and Eric MSP Veith. 2020. Explainable reinforcement learning: A survey. In *International cross-domain conference for machine learning and knowledge extraction*. Springer, 77–95.

Martin L Puterman. 2014. *Markov decision processes: discrete stochastic dynamic programming*. John Wiley & Sons.

Massimo Quadrana, Paolo Cremonesi, and Dietmar Jannach. 2018. Sequence-aware recommender systems. *ACM computing surveys (CSUR)* 51, 4 (2018), 1–36.

Herbert Robbins. 1952. Some aspects of the sequential design of experiments. *Bull. Amer. Math. Soc.* 58, 5 (1952), 527–535. https://doi.org/10.1090/S0002-9904-1952-09620-8

Stuart J Russell and Peter Norvig. 2016. *Artificial intelligence: a modern approach*. pearson.

Daniel J Russo, Benjamin Van Roy, Abbas Kazerouni, Ian Osband, Zheng Wen, et al. 2018. A Tutorial on Thompson Sampling. *Foundations and Trends® in Machine Learning* 11, 1 (2018), 1–96.

Hitesh Sagtani, Madan Gopal Jhawar, Akshat Gupta, and Rishabh Mehrotra. 2023. Quantifying and leveraging user fatigue for interventions in recommender systems. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2293–2297.

Hitesh Sagtani, Madan Gopal Jhawar, Rishabh Mehrotra, and Olivier Jeunen. 2024. Ad-load Balancing via Off-policy Learning in a Content Marketplace. In *Proceedings of the 17th ACM International Conference on Web Search and Data Mining*. 586–595.

Koustuv Saha, Yozen Liu, Nicholas Vincent, Farhan Asif Chowdhury, Leonardo Neves, Neil Shah, and Maarten W Bos. 2021. Advertiming matters: Examining user ad consumption for effective ad allocations on social media. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*. 1–18.

Andrew I. Schein, Alexandrin Popescul, Lyle H. Ungar, and David M. Pennock. 2002. Methods and Metrics for Cold-start Recommendations. In *Proceedings of the 25th annual international ACM SIGIR conference on Research and development in information retrieval (SIGIR '02)*. 253–260. https://doi.org/10.1145/564376.564421

John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz. 2015. Trust region policy optimization. In *International Conference on Machine Learning (ICML)*. PMLR, 1889–1897. https://proceedings.mlr.press/v37/schulman15.pdf

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347* (2017).

Eric M Schwartz, Eric T Bradlow, and Peter S Fader. 2017. Customer acquisition via display advertising using multi-armed bandit experiments. *Marketing Science* 36, 4 (2017), 500–522.

Bobak Shahriari, Kevin Swersky, Ziyu Wang, Ryan P Adams, and Nando De Freitas. 2015. Taking the human out of the loop: A review of Bayesian optimization. *Proc. IEEE* 104, 1 (2015), 148–175.

Sahil Sharma, Aravind Suresh, Rahul Ramesh, and Balaraman Ravindran. 2017. Learning to factor policies and action-value functions: Factored action space representations for deep reinforcement learning. *arXiv preprint arXiv:1705.07269* (2017).

Kyuhong Shim and Wonyong Sung. 2022. A Comparison of Transformer, Convolutional, and Recurrent Neural Networks on Phoneme Recognition. *arXiv preprint arXiv:2210.00367* (2022). https://arxiv.org/abs/2210.00367

Natalia Silberstein, Or Shoham, and Assaf Klein. 2023. Combating Ad Fatigue via Frequency-Recency Features in Online Advertising Systems. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*. 4822–4828.

Wall Street Journal Staff. 2003. Yahoo Agrees to Acquire Overture For $1.63 Billion. *The Wall Street Journal* (2003). https://www.wsj.com/articles/SB105818965623153600

Fei Sun, Jun Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, and Peng Jiang. 2019. BERT4Rec: Sequential recommendation with bidirectional encoder representations from transformer. In *Proceedings of the 28th ACM international conference on information and knowledge management*. 1441–1450.

Richard S. Sutton and Andrew G. Barto. 2018. *Reinforcement Learning: An Introduction* (second ed.). The MIT Press. http://incompleteideas.net/book/the-book-2nd.html

Adith Swaminathan and Thorsten Joachims. 2015. Counterfactual risk minimization: Learning from logged bandit feedback. In *International conference on machine learning*. PMLR, 814–823.

Arash Tavakoli, Fabio Pardo, and Petar Kormushev. 2018. Action branching architectures for deep reinforcement learning. In *Proceedings of the aaai conference on artificial intelligence*, Vol. 32.

Georgios Theocharous, Philip S. Thomas, and Mohammad Ghavamzadeh. 2015. Personalized ad recommendation for lifetime value optimization. In *Proceedings of the 24th International Joint Conference on Artificial Intelligence (IJCAI)*. 1807–1813.

William R. Thompson. 1933. On the likelihood that one unknown probability exceeds another in view of the evidence of two samples. *Biometrika* 25, 3-4 (1933), 285–294.

Bram van den Akker, Olivier Jeunen, Ying Li, Ben London, Zahra Nazari, and Devesh Parekh. 2024. Practical Bandits: An Industry Perspective. In *Proceedings of the 17th ACM International Conference on Web Search and Data Mining*. 1132–1135.

Aaron Van den Oord, Sander Dieleman, and Benjamin Schrauwen. 2013. Deep content-based music recommendation. *Advances in neural information processing systems* 26 (2013).

A Vaswani. 2017. Attention is all you need. *Advances in Neural Information Processing Systems* (2017).

Abhinav Verma, Vijayaraghavan Murali, Rishabh Singh, Pushmeet Kohli, and Swarat Chaudhuri. 2018. Programmatically interpretable reinforcement learning. In *International conference on machine learning*. PMLR, 5045–5054.

Sofía S. Villar, Jack Bowden, and James Wason. 2015. Multi-armed Bandit Models for the Optimal Design of Clinical Trials: Benefits and Challenges. *Statist. Sci.* 30, 2 (2015), 199–215. https://doi.org/10.1214/14-STS504

Elena Voita, David Talbot, Fedor Moiseev, Rico Sennrich, and Ivan Titov. 2019. Analyzing Multi-Head Self-Attention: Specialized Heads Do the Heavy Lifting, the Rest Can Be Pruned. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*. 5797–5808.

Vladislav Vorotilov and Ilnur Shugaepov. 2023. Scaling the Instagram Explore Recommendations System. *Engineering at Meta* (2023). https://engineering.fb.com/2023/08/09/ml-applications/scaling-instagram-explore-recommendations-system/

George A Vouros. 2022. Explainable deep reinforcement learning: state of the art and challenges. *Comput. Surveys* 55, 5 (2022), 1–39.

Yuyan Wang, Mohit Sharma, Can Xu, Sriraj Badam, Qian Sun, Lee Richardson, Lisa Chung, Ed H Chi, and Minmin Chen. 2022. Surrogate for long-term user experience in recommender systems. In *Proceedings of the 28th ACM SIGKDD conference on knowledge discovery and data mining*. 4100–4109.

Ying Wen, Yao Mu, and Martin Ester. 2019. Learning Multi-Objective Rewards and User Utility Function in Recommender Systems. In *Proceedings of the 28th International Joint Conference on Artificial Intelligence (IJCAI)*. International Joint Conferences on Artificial Intelligence Organization, 3719–3725. https://doi.org/10.24963/ijcai.2019/517

Wikipedia contributors. 2025. Cost per mille — Wikipedia, The Free Encyclopedia. https://en.wikipedia.org/wiki/Cost_per_mille [Online; accessed 23-January-2025].

Ronald J. Williams. 1992. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning* 8, 3-4 (1992), 229–256. https://doi.org/10.1007/BF00992696

Qingyun Wu, Hongning Wang, Liangjie Hong, and Yue Shi. 2017. Returning is believing: Optimizing long-term user engagement in recommender systems. In *Proceedings of the 2017 ACM on Conference on Information and Knowledge Management*. 1927–1936.

Siqi Wu, Marian-Andrei Rizoiu, and Lexing Xie. 2018. Beyond views: Measuring and predicting engagement in online videos. In *Proceedings of the International AAAI Conference on Web and Social Media*, Vol. 12.

Xue Xia, Pong Eksombatchai, Nikil Pancha, Dhruvil Deven Badani, Po-Wei Wang, Neng Gu, Saurabh Vishwas Joshi, Nazanin Farahpour, Zhiyuan Zhang, and Andrew Zhai. 2023. Transact: Transformer-based realtime user action model for recommendation at pinterest. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 5249–5259.

Ruiyang Xu, Jalaj Bhandari, Dmytro Korenkevych, Fan Liu, Yuchen He, Alex Nikulkov, and Zheqing Zhu. 2023. Optimizing long-term value for auction-based recommender systems via on-policy reinforcement learning. In *Proceedings of the 17th ACM Conference on Recommender Systems*. 955–962.

Jun Yan, Ning Liu, Gang Wang, Wen Zhang, Yun Jiang, and Zheng Chen. 2009. How much can behavioral targeting help online advertising?. In *Proceedings of the 18th international conference on World wide web*. 261–270.

Jinyun Yan, Zhiyuan Xu, Birjodh Tiwana, and Shaunak Chatterjee. 2020. Ads allocation in feed via constrained optimization. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 3386–3394.

Congrui Yi, David Zumwalt, Zijian Ni, and Shreya Chakrabarti. 2023. Progressive horizon learning: Adaptive long term optimization for personalized recommendation. In *Proceedings of the 17th ACM Conference on Recommender Systems*. 940–946.

Xing Yi, Liangjie Hong, Erheng Zhong, Nanthan Nan Liu, and Suju Rajan. 2014. Beyond clicks: dwell time for personalization. In *Proceedings of the 8th ACM Conference on Recommender systems*. 113–120.

Bowen Yuan, Yaxu Liu, Jui-Yang Hsia, Zhenhua Dong, and Chih-Jen Lin. 2020. Unbiased Ad click prediction for position-aware advertising systems. In *Proceedings of the 14th ACM Conference on Recommender Systems*. 368–377.

Weiru Zhang, Chao Wei, Xiaonan Meng, Yi Hu, and Hao Wang. 2018. The whole-page optimization via dynamic ad allocation. In *Companion Proceedings of the The Web Conference 2018*. 1407–1411.

Jun Zhao, Guang Qiu, Ziyu Guan, Wei Zhao, and Xiaofei He. 2018a. Deep reinforcement learning for sponsored search real-time bidding. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*. 1021–1030.

Xiangyu Zhao, Changsheng Gu, Haoshenglun Zhang, Xiwang Yang, Xiaobing Liu, Jiliang Tang, and Hui Liu. 2021. Dear: Deep reinforcement learning for online advertising impression in recommender systems. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 35. 750–758.

Xiangyu Zhao, Maolin Wang, Xinjian Zhao, Jiansheng Li, Shucheng Zhou, Dawei Yin, Qing Li, Jiliang Tang, and Ruocheng Guo. 2023. Embedding in recommender systems: A survey. *arXiv preprint arXiv:2310.18608* (2023).

Xiangyu Zhao, Liang Zhang, Zhuoye Ding, Long Xia, Jiliang Tang, and Dawei Yin. 2018b. Recommendations with negative feedback via pairwise deep reinforcement learning. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*. 1040–1048.

Xiangyu Zhao, Xudong Zheng, Xiwang Yang, Xiaobing Liu, and Jiliang Tang. 2020. Jointly Learning to Recommend and Advertise. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. ACM, New York, NY, USA, 3319–3327. https://doi.org/10.1145/3394486.3403233

Yu Zhao and Fang Liu. 2024. A survey of retrieval algorithms in ad and content recommendation systems. *arXiv preprint arXiv:2407.01712* (2024).

Zhe Zhao, Lichan Hong, Li Wei, Jilin Chen, Aniruddh Nath, Shawn Andrews, Aditee Kumthekar, Maheswaran Sathiamoorthy, Xinyang Yi, and Ed Chi. 2019. Recommending what video to watch next: a multitask ranking system. In *Proceedings of the 13th ACM conference on recommender systems*. 43–51.

Guorui Zhou, Na Mou, Ying Fan, Qi Pi, Weijie Bian, Chang Zhou, Xiaoqiang Zhu, and Kun Gai. 2019a. Deep Interest Evolution Network for Click-Through Rate Prediction. In *Proceedings of the 33rd AAAI Conference on Artificial Intelligence (AAAI)*. 5941–5948. https://doi.org/10.1609/aaai.v33i01.33015941

Guorui Zhou, Na Mou, Ying Fan, Qi Pi, Weijie Bian, Chang Zhou, Xiaoqiang Zhu, and Kun Gai. 2019b. Deep interest evolution network for click-through rate prediction. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 33. 5941–5948.

Guorui Zhou, Chengru Song, Xiaoqiang Zhu, Ying Fan, Han Zhu, Xiao Ma, Yanghui Yan, Junqi Jin, Han Li, and Kun Gai. 2018. Deep Interest Network for Click-Through Rate Prediction. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD '18)*. 1059–1068. https://doi.org/10.1145/3219819.3219823

Li Zhou and Emma Brunskill. 2016. Latent contextual bandits and their application to personalized recommendations for new users. *arXiv preprint arXiv:1604.06743* (2016).

Jie Zhu, Fengge Wu, and Junsuo Zhao. 2021. An overview of the action space for deep reinforcement learning. In *Proceedings of the 2021 4th International Conference on Algorithms, Computing and Artificial Intelligence*. 1–10.

Yuanyang Zhu, Zhi Wang, Yuanheng Zhu, Chunlin Chen, and Dongbin Zhao. 2024. Discretizing Continuous Action Space With Unimodal Probability Distributions for On-Policy Reinforcement Learning. *IEEE Transactions on Neural Networks and Learning Systems* (2024).

Lixin Zou, Long Xia, Zhuoye Ding, Jiaxing Song, Weidong Liu, and Dawei Yin. 2019. Reinforcement learning to optimize long-term user engagement in recommender systems. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*. 2810–2818.